

FOODRESCUE

Hệ thống web giải cứu thực phẩm cuối ngày

Tài liệu phân tích, thiết kế và kế hoạch dự án

Môn học: Lập trình J2EE

Backend: Spring Boot 3 (REST API) · Frontend: ReactJS · CSDL: MySQL 8

Tích hợp: VNPay Sandbox · Bản đồ OpenStreetMap · OSRM (tính khoảng cách)

4 nhóm người dùng: Quản trị viên · Chủ cửa hàng · Nhân viên cửa hàng · Khách hàng

Phiên bản 4.4 (bản rút gọn, đủ tiền, giỏ hàng, kho hàng; giao diện theo Quy ước chung) · Ngày 25/09/2026

Mục lục

1. Tổng quan dự án	6.1 VNPay Sandbox · 6.2 Bản đồ và tính khoảng cách
1.1 Bối cảnh · 1.2 Giải pháp · 1.3 Mục tiêu · 1.4 Phạm vi	7. Kiến trúc và công nghệ
2. Người dùng và phân quyền	8. Danh sách API
3. Chức năng hệ thống	9. Danh sách màn hình
3.1 Khách hàng · 3.2 Nhân viên · 3.3 Chủ cửa hàng · 3.4 Quản trị viên · 3.5 Tác vụ tự động	10. Thiết kế cơ sở dữ liệu
4. Quy tắc nghiệp vụ	11. Kế hoạch thực hiện
5. Trạng thái, quy trình và dòng tiền	12. Rủi ro
5.5 Hoàn tiền · 5.6 Chi trả theo kỳ · 5.7 Đối soát · 5.8 Sao kê · 5.9 Nhật ký · 5.10 Đối chiếu nghiệp vụ và CSDL	13. Kiểm thử
6. Tích hợp API bên ngoài	14. Tài khoản và kịch bản demo
	Phụ lục A. Script cơ sở dữ liệu
	Phụ lục B. Câu truy vấn kiểm tra dữ liệu

1. Tổng quan dự án

1.1 Bối cảnh

Cuối ngày, tiệm bánh, quán ăn, cửa hàng tiện lợi thường còn dư thức ăn vẫn đảm bảo chất lượng nhưng không bán sang hôm sau được, phần lớn bị bỏ đi. Trong khi đó nhiều người (sinh viên, người đi làm về muộn) sẵn sàng mua với giá rẻ nếu biết chỗ nào gần mình đang còn.

1.2 Giải pháp

FoodRescue là website trung gian. Cửa hàng đăng "**túi giải cứu**" với giá giảm 40-70% kèm khung giờ đến lấy. Khách xem các túi gần mình (có khoảng cách thực tế trên bản đồ), đặt và thanh toán qua **VNPay**, rồi đến quầy đọc **mã nhận hàng**. Nhân viên nhập mã để hoàn tất đơn. Nền tảng giữ lại 15% phí và chuyển phần còn lại cho cửa hàng.

1.3 Mục tiêu

- Website chạy ổn định, đủ 4 vai trò, đi trọn luồng: **đăng túi** → **tìm theo khoảng cách** → **thêm vào giỏ** → **đặt** → **thanh toán VNPay Sandbox** → **nhận hàng bằng mã** → **đánh giá** → **admin chi trả cho cửa hàng**.
- Không bán vượt số lượng khi nhiều người cùng đặt một túi.
- Tiền rõ ràng**: mọi đồng khách trả đều biết đang ở đâu (đã hoàn, đã chi cho cửa hàng, phí nền tảng, hay đang giữ); đối soát khớp tới từng đồng; mọi thao tác tiền đều có nhật ký.
- Hoàn thành trong 8 tuần, code chạy được trên máy cá nhân và có thể deploy lên mạng để demo.

1.4 Phạm vi

Vì đề án chiếm 25% điểm, phạm vi được cắt về **những gì cần để website vận hành được**. Các phần phụ được liệt kê để nhóm có thể nêu là "hướng phát triển" khi báo cáo.

Giữ lại (bắt buộc làm)

- Đăng ký / đăng nhập (JWT), 4 vai trò
- Cửa hàng đăng ký, admin duyệt
- Đăng / sửa / huỷ túi giải cứu, thêm / bớt túi, **sổ kho**
- Tìm túi, bản đồ, **khoảng cách thực tế (OSRM)**
- Giỏ hàng** (nhiều túi của 1 cửa hàng), đặt hàng, **thanh toán VNPay Sandbox**
- Mã nhận hàng (hiện kèm QR), nhân viên nhập mã giao hàng
- Cửa hàng huỷ đơn khi hết hàng; **ghi nhận hoàn tiền** cho từng đơn
- Đánh giá sao sau khi nhận hàng
- Chủ cửa hàng: tài khoản nhân viên, doanh thu, **sao kê từng đơn**
- Admin: duyệt cửa hàng, khoá người dùng, danh mục, đơn hàng, **chi trả theo kỳ, đối soát, nhật ký thao tác**, thống kê
- 2 tác vụ tự động: huỷ đơn chưa thanh toán, hết hạn túi / ghi nhận không đến lấy

Đã lược bỏ (hướng phát triển)

- Xác thực email, quên mật khẩu qua email, gửi email (thay bằng admin đặt lại mật khẩu tạm)
- Refresh token
- Khách tự huỷ đơn sau khi đã thanh toán
- Gọi API hoàn tiền VNPay tự động (admin hoàn thủ công)
- Cửa hàng bấm xác nhận đơn (thanh toán xong là xác nhận luôn)
- Khiếu nại, yêu thích, thông báo trong web
- WebSocket realtime (thay bằng tự tải lại mỗi 15 giây)
- Ví tạm giữ, cửa hàng tự yêu cầu rút tiền (thay bằng admin chi trả theo kỳ)
- Phân quyền chi tiết cho nhân viên, phân ca
- Geocoding (chủ quán tự ghim vị trí trên bản đồ)
- Voucher, cấu hình động, Cloudinary
- Khoá tài khoản khách hay bom hàng

2. Người dùng và phân quyền

Vai trò	Làm được gì
Khách hàng CUSTOMER	Xem và tìm túi gần mình, thêm vào giỏ, đặt hàng, thanh toán VNPay, xem mã nhận hàng, đánh giá. Chưa đăng nhập vẫn xem được danh sách và bản đồ.
Nhân viên STORE_STAFF	Làm việc tại quầy của 1 cửa hàng: xem đơn hôm nay, nhập mã để giao hàng, thêm / bớt túi, xem sổ kho.
Chủ cửa hàng STORE_OWNER	Mọi việc của nhân viên, cộng: hồ sơ cửa hàng, đăng/sửa/hủy túi, hủy đơn, tạo/khoá tài khoản nhân viên, xem doanh thu và số tiền được nhận.
Quản trị viên ADMIN	Duyệt/khoá cửa hàng, khoá người dùng, quản lý danh mục, xem mọi đơn, xác nhận hoàn tiền, tạo và xác nhận kỳ chi trả cho cửa hàng, đối soát tiền, xem nhật ký thao tác, xem thống kê.

Chức năng	Khách	Nhân viên	Chủ CH	Admin
Xem túi, bản đồ, khoảng cách	✓	✓	✓	✓
Giỏ hàng, đặt hàng, thanh toán, đánh giá	✓			
Xem đơn cửa hàng, nhập mã giao hàng, thêm / bớt túi, xem sổ kho		✓	✓	
Đăng/sửa/hủy túi, hủy đơn			✓	
Tài khoản nhân viên, doanh thu			✓	
Duyệt cửa hàng, khoá người dùng, danh mục, hoàn tiền, chi trả, đối soát, nhật ký, thống kê				✓

Phân quyền kiểm tra ở backend bằng Spring Security (`@PreAuthorize` theo vai trò). Chủ cửa hàng và nhân viên chỉ thao tác được trên dữ liệu thuộc cửa hàng của mình.

3. Chức năng hệ thống

3.1 Khách hàng (và khách chưa đăng nhập)

Mã	Chức năng	Mô tả
C-01	Đăng ký, đăng nhập	Đăng ký bằng email, mật khẩu, họ tên, SĐT. Đăng nhập chung cho mọi vai trò, sau đó React chuyển tới trang của vai trò đó. Tài khoản đang dùng mật khẩu tạm (nhân viên mới, vừa được admin đặt lại) bị chuyển tới trang đổi mật khẩu trước.
C-02	Trang chủ, danh sách túi	Hiện túi đang bán; lọc theo danh mục, bán kính (1/3/5/10 km); sắp xếp gần nhất / rẻ nhất / sắp hết giờ. Mỗi túi hiện "2,3 km · 8 phút".
C-03	Bản đồ quanh tôi	Lấy vị trí bằng trình duyệt (hoặc bấm chọn trên bản đồ); hiện marker các cửa hàng có túi đang bán.
C-04	Chi tiết túi	Ảnh, mô tả, giá gốc / giá giải cứu, số còn lại, khung giờ lấy, địa chỉ, khoảng cách, điểm đánh giá cửa hàng; chọn số lượng và nút Thêm vào giỏ .
C-05a	Giỏ hàng	Thêm túi vào giỏ (mỗi loại 1-5 cái), sửa số lượng, xoá. Giỏ chỉ chứa túi của một cửa hàng ; thêm túi của quán khác thì hỏi "Xoá giỏ hiện tại?". Giỏ lưu trong CSDL (đổi máy vẫn còn) nhưng không giữ hàng ; mở giỏ thì túi đã hết hoặc hết giờ được đánh dấu và không đặt được.
C-05b	Đặt hàng và thanh toán	Từ giỏ bấm "Đặt hàng": hệ thống kiểm tra lại từng túi, trừ kho tất cả hoặc không túi nào (thiếu 1 túi thì báo túi nào thiếu), tính khung giờ lấy chung, tạo đơn + chi tiết đơn, xoá giỏ, chuyển sang VNPay, quay lại trang kết quả. Đơn chưa thanh toán thì khách tự huỷ được (số lượng được trả lại ngay).
C-06	Đơn của tôi	Danh sách đơn và trạng thái; đơn đã thanh toán hiện mã nhận hàng + QR , khung giờ lấy, nút "Chỉ đường" (mở Google Maps); mỗi đơn hiện số tiền đã trả; đơn bị huỷ hiện trạng thái hoàn tiền (đang hoàn / đã hoàn + mã giao dịch).
C-07	Đánh giá	Sau khi nhận hàng: chấm 1-5 sao và bình luận, mỗi đơn 1 lần.
C-08	Hồ sơ	Sửa họ tên, SĐT, đổi mật khẩu.

3.2 Nhân viên cửa hàng

Mã	Chức năng	Mô tả
ST-01	Màn hình quầy	Đơn hôm nay của cửa hàng, nhóm theo trạng thái; tự tải lại mỗi 15 giây để thấy đơn mới.
ST-02	Giao hàng bằng mã	Nhập mã 6 ký tự khách đọc → hiện tên khách và danh sách túi, số lượng → bấm "Đã giao" → đơn hoàn tất.
ST-03	Thêm / bớt túi	Thêm túi (làm thêm được) hoặc bớt túi (khách tại quầy mua mất). Hệ thống cộng/trừ đồng thời <code>quantity</code> và <code>remaining_quantity</code> (BR-18); không bớt quá số còn lại.
ST-04	Sổ kho	Xem lịch sử tăng/giảm của từng túi: đăng, thêm, bớt, khách đặt, trả lại; ai làm, lúc nào, còn lại bao nhiêu.

3.3 Chủ cửa hàng

Mã	Chức năng	Mô tả
O-01	Đăng ký cửa hàng	Tạo tài khoản chủ + thông tin cửa hàng, ghim vị trí trên bản đồ , giờ mở cửa, tài khoản ngân hàng, ảnh giấy phép VSATTP. Chờ admin duyệt.
O-02	Hồ sơ cửa hàng	Sửa thông tin, ảnh, vị trí, giờ mở cửa, tài khoản ngân hàng.

Mã	Chức năng	Mô tả
O-03	Quản lý túi giải cứu	Tạo / sửa / huỷ túi: tiêu đề, danh mục, ảnh, mô tả, giá gốc, giá giải cứu, số lượng, khung giờ lấy. Túi đã có đơn thì không sửa giá và khung giờ (đơn cũ vẫn giữ giá lúc mua). Mọi lần đăng, thêm, bớt đều ghi vào sổ kho.
O-04	Quản lý đơn	Xem đơn theo ngày và trạng thái; giao hàng bằng mã; huỷ đơn khi hết hàng (phải ghi lý do, đơn chuyển sang chờ hoàn tiền); chi tiết đơn liệt kê từng túi, số lượng, giá lúc mua.
O-05	Nhân viên	Tạo tài khoản nhân viên (email + mật khẩu tạm, nhân viên phải đổi ở lần đăng nhập đầu), khoá / mở khoá.
O-06	Doanh thu và sao kê	Theo khoảng ngày: số đơn, số túi bán, doanh thu, phí, thực nhận; biểu đồ theo ngày; sao kê từng đơn (đã chi ở kỳ nào / chưa chi); danh sách kỳ chi trả kèm mã chuyển khoản; tổng tiền chưa được chi; xem đánh giá.
O-07	Yêu cầu rút tiền	Bấm "Yêu cầu rút tiền": hệ thống gom mọi đơn chưa chi đến hôm nay thành một kỳ chi trả <code>PENDING</code> gửi admin; theo dõi đến khi admin chuyển khoản (có mã giao dịch).

3.4 Quản trị viên

Mã	Chức năng	Mô tả
A-01	Thống kê	Tổng đơn, doanh thu (GMV), phí thu được, số túi đã giải cứu, số cửa hàng / khách hàng; biểu đồ theo ngày.
A-02	Duyệt cửa hàng	Xem thông tin, ảnh giấy phép, vị trí trên bản đồ → duyệt (đặt ti lệ phí , mặc định 15%, sửa được sau) hoặc từ chối (ghi lý do). Khoá / mở khoá cửa hàng: khi khoá thì túi bị ẩn, còn đơn đã thanh toán vẫn phải được giao hoặc admin huỷ và hoàn tiền.
A-03	Người dùng	Danh sách, tìm theo email/vai trò, khoá / mở khoá; đặt lại mật khẩu tạm khi người dùng quên (người dùng bị bắt đổi ở lần đăng nhập sau).
A-04	Danh mục	Thêm / sửa / xoá danh mục túi.
A-05	Đơn hàng và hoàn tiền	Xem mọi đơn, tra theo mã; chi tiết đơn có lịch sử tiền (thanh toán, hoàn tiền, kỳ chi trả). Trang Hoàn tiền: chờ hoàn / đã hoàn; admin hoàn trên VNPAY Sandbox rồi nhập mã giao dịch hoàn để xác nhận (mục 5.5).
A-06	Chi trả theo kỳ	Bảng tiền chưa chi của từng cửa hàng; tạo kỳ chi trả (chọn cửa hàng + khoảng ngày, hệ thống gom đơn và tính tiền); chuyển khoản rồi nhập mã chuyển khoản; xử lý các yêu cầu rút tiền cửa hàng gửi lên; xem chi tiết từng kỳ (mục 5.6).
A-07	Đối soát tổng	Trang đối soát theo khoảng ngày: 7 khoản tiền và dòng kiểm tra chênh lệch = 0 (mục 5.7); bảng theo từng cửa hàng.
A-08	Nhật ký thao tác	Danh sách nhật ký, lọc theo hành động, loại đối tượng, người thực hiện, ngày; bấm vào để mở đơn / kỳ chi trả / cửa hàng liên quan.

3.5 Tác vụ tự động

Tác vụ	Chu kỳ	Việc làm
Huỷ đơn chưa thanh toán	1 phút	Đơn <code>PENDING_PAYMENT</code> quá 15 phút → <code>EXPIRED</code> , cộng trả số lượng cho các túi trong đơn (ghi sổ kho <code>RETURNED</code>). (Link VNPAY hết hạn ở phút 10; chờ thêm 5 phút cho kết quả thanh toán về muộn.)
Hết hạn túi, không đến lấy	5 phút	Túi quá <code>pickup_end</code> → <code>EXPIRED</code> . Đơn <code>CONFIRMED</code> quá <code>pickup_end</code> + 30 phút → <code>NO_SHOW</code> (không hoàn tiền, cửa hàng vẫn nhận tiền).

4. Quy tắc nghiệp vụ

Mã	Quy tắc
BR-01	Email là duy nhất; mật khẩu ≥ 6 ký tự, lưu mã hoá BCrypt. Tài khoản bị khoá không đăng nhập được. Tài khoản có <code>must_change_password = TRUE</code> chỉ gọi được API đổi mật khẩu cho đến khi đổi xong.
BR-02	Cửa hàng chỉ được đăng túi và hiện lên trang khách khi đã được admin duyệt (<code>APPROVED</code>). Mỗi tài khoản chủ có đúng 1 cửa hàng; mỗi nhân viên thuộc đúng 1 cửa hàng.
BR-03	Giá giảm cứu > 0 và $\leq 60\%$ giá gốc (giảm tối thiểu 40%).
BR-04	Khung giờ lấy: giờ kết thúc sau giờ bắt đầu, và giờ kết thúc chưa qua.
BR-05	Mỗi loại túi trong giỏ / đơn có 1-5 cái; không được bán vượt số còn lại (mục 7.4); đơn nhiều túi thì trừ kho tất cả hoặc không trừ túi nào. Hết hàng thì túi chuyển <code>SOLD_OUT</code> .
BR-06	Đặt hàng thì trừ số lượng ngay; link VNPay hết hạn sau 10 phút ; quá 15 phút chưa thanh toán thì đơn hết hạn và trả số lượng. Khách tự huỷ được đơn chưa thanh toán.
BR-07	Thanh toán thành công (chữ ký VNPay hợp lệ, số tiền khớp) thì đơn chuyển thẳng sang <code>CONFIRMED</code> .
BR-08	Khách không tự huỷ đơn đã thanh toán. Chỉ chủ cửa hàng (hết hàng) hoặc admin được huỷ, và chỉ huỷ được đơn <code>CONFIRMED</code> (chưa giao). Khi huỷ: tạo 1 bản ghi hoàn tiền bằng đúng <code>total_amount</code> , đơn chuyển <code>REFUND_PENDING</code> , số lượng được trả lại túi.
BR-09	Chỉ giao hàng khi đơn <code>CONFIRMED</code> , đúng cửa hàng, trong khoảng từ 15 phút trước giờ bắt đầu lấy đến 30 phút sau giờ kết thúc.
BR-10	Khách không đến lấy (quá giờ kết thúc 30 phút) thì đơn là <code>NO_SHOW</code> , không hoàn tiền.
BR-11	Chỉ đánh giá đơn <code>COMPLETED</code> , mỗi đơn 1 lần.
BR-12	Tỉ lệ phí của cửa hàng (<code>stores.commission_rate</code> , mặc định 15%, admin đặt) được copy vào đơn lúc đặt (<code>orders.commission_rate</code>): $commission = round(total \times rate / 100)$, $store_earning = total - commission$. Đổi tỉ lệ chỉ áp dụng cho đơn mới.
BR-13	Đơn <code>COMPLETED</code> hoặc <code>NO_SHOW</code> không huỷ, không hoàn được; số tiền cửa hàng nhận của các đơn này là cố định.
BR-14	Chỉ đơn <code>COMPLETED</code> / <code>NO_SHOW</code> mới được đưa vào kỳ chi trả; mỗi đơn thuộc tối đa 1 kỳ; <code>net_amount</code> của kỳ = tổng <code>store_earning</code> của đúng các đơn trong kỳ.
BR-15	Kỳ chi trả chỉ chuyển <code>PAID</code> khi có mã chuyển khoản; kỳ đã <code>PAID</code> không sửa, không huỷ. Tài khoản ngân hàng nhận tiền được chụp lại vào kỳ lúc tạo.
BR-16	Hoàn tiền chỉ chuyển <code>COMPLETED</code> khi có mã giao dịch hoàn; mỗi đơn hoàn tối đa 1 lần.
BR-17	Mọi thao tác tiền và quản trị đều ghi <code>audit_logs</code> . Không xoá đơn, thanh toán, hoàn tiền, kỳ chi trả hay nhật ký.
BR-18	<code>quantity</code> là tổng số túi đã đưa lên app, <code>remaining_quantity</code> là số còn lại. Thêm hoặc bớt túi thì cộng/trừ cả hai , để <code>quantity - remaining_quantity</code> luôn bằng số túi đã bán qua app.
BR-19	Thanh toán về khi đơn đã hết hạn: nếu túi còn hàng thì trừ lại số lượng và xác nhận đơn; nếu hết thì huỷ đơn và tự tạo yêu cầu hoàn tiền (lý do <code>LATE_PAYMENT</code>). Khách không bao giờ mất tiền mà không có hàng.
BR-20	Cửa hàng yêu cầu rút tiền khi có ít nhất 1 đơn chưa chi và không có kỳ nào đang <code>PENDING</code> ; kỳ được tạo gồm mọi đơn chưa chi đến hôm nay.
BR-21	Giỏ hàng chỉ chứa túi của 1 cửa hàng, không giữ hàng; giá và số còn lại luôn lấy mới nhất khi mở giỏ và khi đặt. Đặt thành công thì giỏ được xoá.
BR-22	Khung giờ lấy của đơn = phần giao nhau khung giờ các túi trong đơn (bắt đầu muộn nhất đến kết thúc sớm nhất); không có phần giao nhau thì không cho đặt chung.

Mã	Quy tắc
BR-23	Mọi thay đổi số lượng túi (đăng, thêm, bớt, đặt, trả lại) chỉ đi qua một hàm dùng chung và ghi 1 dòng <code>stock_movements</code> ; tổng các dòng luôn bằng số còn lại.
BR-24	Hủy túi khi còn đơn: đơn <code>CONFIRMED</code> có túi đó được hủy cả đơn và hoàn toàn bộ (<code>BATCH_CANCELLED</code>); đơn <code>PENDING_PAYMENT</code> có túi đó chuyển <code>CANCELLED</code> + <code>UNPAID</code> và trả kho; nếu tiền VNPay của đơn đó vẫn về sau thì xử lý như BR-19 (tự hoàn). Túi trong giỏ hàng bị đánh dấu "không còn bán".
BR-25	Khoá cửa hàng: túi bị ẩn, không thêm vào giỏ và không đặt được; đơn <code>CONFIRMED</code> vẫn phải được giao hoặc admin hủy + hoàn; đơn chưa thanh toán hết hạn như bình thường; tiền đã bán vẫn được chi trả.
BR-26	Múi giờ: mọi thời gian lưu và so sánh theo giờ Việt Nam (<code>Asia/Ho_Chi_Minh</code> , +07:00) ở cả MySQL, JDBC, JVM và JSON trả về.

5. Trạng thái, quy trình và dòng tiền

5.1 Trạng thái đơn hàng

PENDING_PAYMENT → VNPay thành công → CONFIRMED → nhập mã giao hàng → COMPLETED

PENDING_PAYMENT → quá 10 phút / huỷ thanh toán → EXPIRED

| CONFIRMED → cửa hàng / admin huỷ → CANCELLED | CONFIRMED → quá giờ → NO_SHOW

Trạng thái thanh toán đi kèm: UNPAID → PAID ; khi huỷ đơn đã trả tiền: PAID → REFUND_PENDING → REFUNDED (admin xác nhận).

5.2 Trạng thái túi và cửa hàng

AVAILABLE ⇔ hết / có đơn trả lại ⇔ SOLD_OUT → quá giờ → EXPIRED | chủ huỷ → CANCELLED

PENDING → admin duyệt → APPROVED ⇔ LOCKED | PENDING → REJECTED

5.3 Quy trình giỏ hàng, đặt hàng và thanh toán

- Khách thêm các túi của một cửa hàng vào giỏ, mở giỏ, bấm "Đặt hàng". React gọi `POST /api/orders` (backend lấy dữ liệu từ giỏ của khách, không tin giá gửi lên).
- Backend, trong **1 transaction**: kiểm tra từng túi còn bán và khung giờ còn giao nhau; trừ kho từng túi bằng `UPDATE` có điều kiện (sắp theo id túi để tránh deadlock) và ghi sổ kho `ORDERED`; thiếu 1 túi thì rollback toàn bộ; tạo `orders + order_items` (giá chụp lại), mã nhận hàng, `payments` `PENDING`; xoá giỏ; trả về link VNPay.
- React chuyển trình duyệt sang VNPay. Khách nhập thẻ test và OTP.
- VNPay chuyển về trang `/payment/result` của React kèm kết quả. React gửi toàn bộ tham số về backend; backend kiểm tra chữ ký, số tiền rồi cập nhật đơn sang `CONFIRMED`. Nếu có IPN (khi deploy hoặc dùng ngrok), IPN cũng gọi cùng một hàm xử lý, và hàm này không xử lý lại đơn đã xác nhận.
- Trang kết quả hiện "Đặt thành công" và mã nhận hàng.

5.4 Quy trình nhận hàng

- Khách đến quầy trong khung giờ, mở "Đơn của tôi" và đọc mã (hoặc đưa QR).
- Nhân viên nhập mã → backend kiểm tra (BR-09) → hiện tên khách và danh sách túi trong đơn.
- Nhân viên đưa túi, bấm "Đã giao" → đơn `COMPLETED`, ghi người giao. Khách thấy nút "Đánh giá".

5.5 Hoàn tiền cho khách

- Khi nào**: chủ cửa hàng huỷ đơn (hết hàng), chủ cửa hàng huỷ cả túi, hoặc admin huỷ đơn; chỉ huỷ được đơn `CONFIRMED` (đã trả tiền, chưa giao), khách không tự huỷ đơn đã trả tiền. Ngoài ra hệ thống tự tạo yêu cầu hoàn khi **tiền về muộn** mà túi đã hết (BR-19, lý do `LATE_PAYMENT`).
- Hoàn bao nhiêu**: luôn hoàn **toàn bộ** `total_amount`. Mỗi đơn hoàn tối đa một lần.

Huỷ đơn (lý do) → Đơn **CANCELLED + REFUND_PENDING**, trả số lượng, tạo **refunds** `PENDING` →

Admin hoàn trên VNPay Sandbox → Nhập mã giao dịch hoàn: **refunds** `COMPLETED`, đơn `REFUNDED`

- Mọi chỗ huỷ đơn đều gọi chung một hàm `RefundService.request(order, reason)`. Trong 1 transaction, hàm này đổi trạng thái đơn, trả số lượng cho các túi trong đơn (ghi sổ kho `RETURNED`), tạo bản ghi `refunds` (mã `RF-<mã đơn>`, số tiền, lý do, người huỷ) và ghi nhật ký.
- Admin mở trang **Hoàn tiền**, thấy danh sách chờ hoàn kèm mã giao dịch VNPay gốc (`payments.gateway_txn_no`), thực hiện hoàn trên trang quản lý merchant VNPay Sandbox.
- Admin nhập **mã giao dịch hoàn** rồi bấm "Xác nhận đã hoàn". Hệ thống không cho xác nhận nếu thiếu mã này.

5.6 Chi trả cho cửa hàng theo kỳ

Admin chi trả theo **kỳ** (ví dụ mỗi thứ Hai chi cho tuần trước). Mỗi kỳ chi trả **gắn với danh sách đơn cụ thể** (cột `orders.payout_id`), nên luôn trả lời được "khoản tiền này là của những đơn nào" và một đơn không bao giờ bị chi hai lần.

Chọn cửa hàng + khoảng ngày → Xem trước danh sách đơn và tổng tiền →
Tạo kỳ PENDING (gắn đơn vào kỳ) → Chuyển khoản ngân hàng → Nhập mã chuyển khoản: PAID

- Đơn đủ điều kiện:** thuộc cửa hàng đó, trạng thái `COMPLETED` hoặc `NO_SHOW`, `completed_at` nằm trong kỳ, **chưa thuộc kỳ nào** (`payout_id IS NULL`).
- Tạo kỳ:** hệ thống tính `order_count`, `gross_amount` (tổng khách trả), `commission_amount` (tổng phí), `net_amount = gross - commission`; chụp lại tài khoản ngân hàng hiện tại của cửa hàng; gắn `payout_id` cho các đơn bằng `UPDATE ... WHERE payout_id IS NULL`. Hai admin bấm cùng lúc thì đơn vẫn chỉ vào một kỳ.
- Xác nhận đã chi:** admin chuyển đúng `net_amount` vào tài khoản đã chụp, nhập mã giao dịch ngân hàng → `PAID`. Kỳ đã `PAID` không sửa và không hủy được.
- Nhập sai:** kỳ còn `PENDING` thì hủy được (`CANCELLED`), các đơn được gỡ khỏi kỳ để gom lại sau.

Cửa hàng yêu cầu rút tiền: chủ cửa hàng bấm "Yêu cầu rút tiền" → hệ thống dùng cùng hàm tạo kỳ ở bước 2 với khoảng ngày từ đơn chưa chi cũ nhất đến hôm nay, `created_by` là chủ cửa hàng → kỳ `PENDING` hiện trong danh sách của admin → admin chuyển khoản và nhập mã như bước 3. Không cần bảng riêng cho yêu cầu rút tiền.

5.7 Công thức đối soát

Mọi đồng tiền khách đã trả phải nằm ở đúng một trong các khoản sau. Trang **Đối soát** của admin tính từng khoản từ CSDL và hiện dòng kiểm tra; **chênh lệch phải bằng 0**, khác 0 là có lỗi dữ liệu.

#	Khoản	Cách tính	Dữ liệu mẫu
A	Tiền thu từ khách	Tổng <code>payments.amount</code> có <code>status = SUCCESS</code>	464.000
1	Đã hoàn cho khách	Tổng <code>refunds.amount</code> , <code>status = COMPLETED</code>	0
2	Đang chờ hoàn	Tổng <code>refunds.amount</code> , <code>status = PENDING</code>	45.000
3	Đã chi cho cửa hàng	Tổng <code>payouts.net_amount</code> , <code>status = PAID</code>	170.000
4	Đang chờ chi (kỳ PENDING)	Tổng <code>payouts.net_amount</code> , <code>status = PENDING</code>	0
5	Còn nợ cửa hàng (chưa vào kỳ)	Tổng <code>store_earning</code> đơn <code>COMPLETED/NO_SHOW</code> có <code>payout_id IS NULL</code>	18.700
6	Phí nền tảng đã hưởng	Tổng <code>commission_amount</code> đơn <code>COMPLETED/NO_SHOW</code>	33.300
7	Tiền đơn chưa giao	Tổng <code>total_amount</code> đơn <code>CONFIRMED</code> (chưa biết sẽ giao hay hủy)	197.000
	Kiểm tra	$A - (1 + 2 + 3 + 4 + 5 + 6 + 7)$	0

Số dư tài khoản nền tảng lúc này = $A - 1 - 3$ (tiền đã thu trừ tiền đã trả ra). Trong đó, phần thực sự là doanh thu của nền tảng chỉ là khoản 6; các khoản 2, 4, 5, 7 là tiền đang giữ hộ khách hoặc cửa hàng.

5.8 Sao kê cho cửa hàng

Trang Doanh thu của chủ cửa hàng có **sao kê từng đơn**, mỗi dòng: mã đơn, ngày hoàn tất, khách trả, tỉ lệ phí, phí, thực nhận, trạng thái (đã chi ở kỳ `PO-...` hoặc chưa chi). Bên dưới là danh sách các kỳ chi trả (số đơn, số tiền, tài khoản nhận, mã chuyển khoản, ngày chi). Chủ cửa hàng tự đối chiếu được với sao kê ngân hàng của mình.

5.9 Nhật ký thao tác

Bảng `audit_logs` ghi lại mọi thao tác liên quan đến tiền và quản trị: **ai** (hoặc hệ thống), **làm gì**, **trên đối tượng nào**, **số tiền**, **lúc nào**. Chỉ thêm, không sửa, không xóa: CSDL có trigger chặn mọi lệnh UPDATE / DELETE trên bảng này (sổ kho `stock_movements` cũng vậy).

Nhóm	Hành động được ghi
Thanh toán, đơn hàng	<code>PAYMENT_SUCCESS</code> (hệ thống), <code>ORDER_CANCELLED</code> , <code>ORDER_EXPIRED</code> , <code>ORDER_COMPLETED</code> , <code>ORDER_NO_SHOW</code>
Hoàn tiền	<code>REFUND_REQUESTED</code> , <code>REFUND_COMPLETED</code>
Chi trả	<code>PAYOUT_CREATED</code> , <code>PAYOUT_PAID</code> , <code>PAYOUT_CANCELLED</code>
Quản trị	<code>BATCH_CANCELLED</code> , <code>STORE_APPROVED</code> , <code>STORE_REJECTED</code> , <code>STORE_LOCKED</code> , <code>STORE_RATE_CHANGED</code> , <code>USER_LOCKED</code> , <code>USER_UNLOCKED</code> , <code>USER_PASSWORD_RESET</code>

Backend có một hàm dùng chung `AuditService.log(action, entityType, entityId, amount, detail)`, tự lấy người thực hiện từ JWT; được gọi trong cùng transaction với thao tác chính.

5.10 Đối chiếu nghiệp vụ và CSDL

Bảng dưới kiểm tra từng nghiệp vụ cơ bản đã có đủ chỗ lưu trong CSDL và có ràng buộc hoặc câu kiểm tra đi kèm. File `foodrescue_checks.sql` chứa 18 câu truy vấn (C0-C13, thêm C1b, C1c, C3b, C3c, C3d): chạy sau khi test hoặc trước khi demo; dữ liệu đúng thì các câu C1-C12 trả về 0 dòng và C0 có chênh lệch bằng 0. Đã chạy trên **MySQL 8.0.40** (chế độ strict mặc định) và MariaDB 10.4 của XAMPP: dữ liệu mẫu sạch; đã thử cố ý làm sai tồn kho, tiền giao dịch, tổng đơn, giỏ hàng: các câu tương ứng đều báo lỗi.

Nghệp vụ	Lưu ở đâu	Ràng buộc trong CSDL / câu kiểm tra
Đăng bán và tồn kho		
Đăng bán túi	<code>rescue_batches</code> : giá gốc, giá bán, số lượng, khung giờ, danh mục, ảnh	<code>chk_batch_price</code> (giảm $\geq 40\%$), <code>chk_batch_time</code> , <code>chk_batch_qty</code>
Thêm / bớt túi	<code>quantity</code> và <code>remaining_quantity</code> cùng cộng/trừ (BR-18)	<code>chk_batch_qty</code> ; C1: <code>quantity - remaining =</code> số túi đã bán qua app; C2: trạng thái <code>SOLD_OUT</code> đúng với số còn lại
Sổ kho	<code>stock_movements</code> (LISTED, ADDED, REMOVED, ORDERED, RETURNED)	Trigger chặn sửa / xóa; <code>chk_stock_sign</code> (dấu đúng với lý do), <code>chk_stock_order</code> ; C1b: tổng biến động = số còn lại; C1c: mỗi túi trong đơn có đúng dòng trừ / trả kho
Không bán vượt	UPDATE có điều kiện <code>remaining_quantity >= :qty</code>	<code>chk_batch_qty</code> (không âm); test 10 luồng (TC01)
Đặt hàng và thanh toán		
Giỏ hàng	<code>cart_items</code> (khóa chính khách + túi)	<code>chk_cart_qty</code> (1-5); C3d: giỏ chỉ có túi của 1 cửa hàng
Đơn nhiều túi	<code>order_items</code> (giá chụp lại lúc mua)	<code>chk_items_money</code> , <code>uq_items_batch</code> ; C3b: tổng đơn = tổng chi tiết; C3c: khung giờ đơn nằm trong khung giờ mọi túi
Tiền khách trả	<code>order_items.unit_price</code> (giá lúc mua), <code>orders.total_amount</code> ; <code>payments</code> (giao dịch VNPAY)	<code>chk_order_money</code> ; C4: đơn đã trả có đúng 1 giao dịch SUCCESS bằng đúng số tiền

Nghệp vụ	Lưu ở đâu	Ràng buộc trong CSDL / câu kiểm tra
Trạng thái đơn và thanh toán	<code>orders.status + payment_status</code>	C5: hai trạng thái phải khớp nhau; <code>chk_order_done_time</code> : COMPLETED/NO_SHOW phải có thời điểm chốt
Đơn đúng cửa hàng	<code>orders.store_id , order_items.batch_id</code>	C3: mọi túi trong đơn thuộc cửa hàng của đơn
Giao hàng	<code>orders.handled_by , completed_at</code>	C11: người giao là chủ hoặc nhân viên của đúng cửa hàng
Tiền của nền tảng và cửa hàng		
% phí của admin	<code>stores.commission_rate</code> (admin đặt) → copy vào <code>orders.commission_rate ; commission_amount</code>	<code>chk_store_rate</code> (0-50%); <code>chk_order_money</code> : phí = $\text{round}(\text{total} \times \text{rate} / 100)$
Tiền cửa hàng nhận	<code>orders.store_earning</code>	<code>chk_order_money</code> : thực nhận = total – phí
Rút tiền / chi trả	<code>payouts</code> (admin tạo theo kỳ hoặc cửa hàng yêu cầu) + <code>orders.payout_id</code>	<code>chk_payout_money</code> , <code>chk_payout_paid</code> , <code>chk_order_payout</code> ; C7: số tiền kỳ = tổng các đơn trong kỳ; C8, C9
Hoàn tiền	<code>refunds</code> (1 đơn tối đa 1 lần)	<code>refunds.order_id</code> UNIQUE, <code>chk_refund_done</code> ; C6: đơn chờ hoàn/đã hoàn khớp bản ghi hoàn tiền
Đối soát	Tính từ <code>payments</code> , <code>refunds</code> , <code>payouts</code> , <code>orders</code>	C0: chênh lệch = 0
Nhật ký	<code>audit_logs</code>	Trigger chặn sửa / xoá; C12: thanh toán, hoàn tiền, chi trả đều có dòng nhật ký
Mật khẩu tạm	<code>users.must_change_password</code>	Kiểm tra ở backend khi đăng nhập / gọi API (BR-01)
Thời gian	Mọi cột DATETIME theo giờ Việt Nam	Cấu hình múi giờ mục 7.6 (BR-26)
Tài khoản		
Chủ và nhân viên	<code>stores.owner_id</code> UNIQUE; <code>users.store_id</code>	<code>chk_users_staff</code> : nhân viên bắt buộc có cửa hàng; một chủ chỉ một cửa hàng
Đánh giá	<code>reviews</code> (1 đơn 1 đánh giá)	C10: chỉ đơn COMPLETED, đúng khách, đúng cửa hàng

6. Tích hợp API bên ngoài

Nhu cầu	Dịch vụ	Ghi chú
Thanh toán	VNPay Sandbox	Miễn phí, đăng ký tài khoản merchant test. Gọi từ backend.
Tính khoảng cách, thời gian	OSRM Table API	Không cần key. Gọi từ backend, có dự phòng khi lỗi.
Hiển thị bản đồ	Leaflet + OpenStreetMap	Không cần key. Dùng thư viện react-leaflet.
Vị trí người dùng	Geolocation API của trình duyệt	Chỉ chạy trên HTTPS hoặc localhost. Người dùng từ chối thì bấm chọn trên bản đồ.
Chỉ đường	Link Google Maps	<code>https://www.google.com/maps/dir/?api=1&destination={lat},{lng}</code> , không cần key.

6.1 VNPay Sandbox

a) Chuẩn bị

- Đăng ký merchant test trên trang sandbox của VNPay, nhận `vnp_TmnCode` và `vnp_HashSecret` qua email.
- URL thanh toán: `https://sandbox.vnpayment.vn/paymentv2/vpcpay.html`
- Key đặt trong biến môi trường, không đưa lên GitHub.

b) Tạo link thanh toán

Tham số	Giá trị
<code>vnp_Version</code> , <code>vnp_Command</code>	2.1.0, pay
<code>vnp_TmnCode</code>	Mã website sandbox
<code>vnp_Amount</code>	Số tiền × 100 (45.000đ → 4500000)
<code>vnp_CurrCode</code> , <code>vnp_Locale</code>	VND, vn
<code>vnp_TxnRef</code>	Mã giao dịch duy nhất (<code>payments.txn_ref</code>)
<code>vnp_OrderInfo</code> , <code>vnp_OrderType</code>	"Thanh toan don FR260925-0004", other
<code>vnp_ReturnUrl</code>	<code>http://localhost:5173/payment/result</code> (trang React)
<code>vnp_IpAddr</code>	IP của khách
<code>vnp_CreateDate</code> , <code>vnp_ExpireDate</code>	yyyyMMddHHmmss giờ Việt Nam; hết hạn sau 10 phút
<code>vnp_SecureHash</code>	HMAC-SHA512 (khóa <code>vnp_HashSecret</code>) của chuỗi tham số đã sắp xếp theo tên, URL-encode, nối bằng &

c) Xác nhận kết quả

```
// PaymentService.confirm(params) - dùng chung cho Return URL và IPN
@Transactional
public Order confirm(Map<String, String> params) {
    if (!vnpaySigner.verify(params)) throw new BusinessException("Sai chữ ký VNPay");
    Payment p = paymentRepo.findByTxnRef(params.get("vnp_TxnRef")).orElseThrow();
    if (p.getAmount().longValue() * 100 != Long.parseLong(params.get("vnp_Amount")))
        throw new BusinessException("Sai số tiền");
    if (p.getStatus() != PaymentStatus.PENDING) return p.getOrder(); // đã xử lý rồi -> bỏ qua
    if ("00".equals(params.get("vnp_ResponseCode"))) {
        p.markSuccess(params); // SUCCESS, paid_at, gateway_txn_no, bank_code
        p.getOrder().markPaid(); // CONFIRMED + PAID
    } else {
        p.markFailed(params.get("vnp_ResponseCode"));
        orderService.expireAndReturnStock(p.getOrder()); // EXPIRED, cộng lại số lượng
    }
    return p.getOrder();
}
```

Mã `vnp_ResponseCode`: `00` thành công, `24` khách huỷ giao dịch, các mã khác là thất bại. Khi chạy trên máy cá nhân, VNPay không gọi được IPN về `localhost`, nên hệ thống xác nhận qua Return URL (vẫn kiểm tra chữ ký). Khi deploy thì khai báo thêm IPN URL `/api/payments/vnpay/ipn`.

d) Thử test

Ngân hàng **NCB** · Số thẻ `9704198526191432198` · Tên **NGUYEN VAN A** · Ngày phát hành **07/15** · OTP `123456`.
(Theo tài liệu sandbox công khai của VNPay; kiểm tra lại khi tích hợp.)

6.2 Bản đồ và tính khoảng cách

a) Cách tính

Bước	Cách làm
1. Lọc	MySQL lọc cửa hàng đã duyệt, có túi đang bán, trong bán kính R (đường chim bay, hàm <code>ST_Distance_Sphere</code>); lấy tối đa 30 cửa hàng gần nhất.
2. Khoảng cách thật	Gọi 1 lần OSRM Table API cho cả 30 cửa hàng để lấy quãng đường (mét) và thời gian (giây) theo đường đi thực tế; sắp xếp lại.
3. Dự phòng	OSRM lỗi hoặc quá 2 giây: dùng khoảng cách chim bay $\times 1,3$, đánh dấu <code>estimated = true</code> , React hiện <code>"~2,4 km"</code> . Website vẫn chạy bình thường.

```
-- Bước 1 (kết quả tính bằng mét)
SELECT s.*, ST_Distance_Sphere(POINT(s.longitude, s.latitude), POINT(:lng, :lat)) AS air_m
FROM stores s
WHERE s.status = 'APPROVED'
      AND EXISTS (SELECT 1 FROM rescue_batches b WHERE b.store_id = s.id
                  AND b.status = 'AVAILABLE' AND b.pickup_end > NOW())
HAVING air_m <= :radiusM
ORDER BY air_m LIMIT 30;

-- Bước 2: OSRM (thứ tự toạ độ là KINH ĐỘ, VĨ ĐỘ; điểm 0 là vị trí khách)
GET http://router.project-osrm.org/table/v1/driving/106.69,10.77;106.671239,10.758372;106.698942,10.781254
?sources=0&annotations=distance,duration
→ { "distances": [[0, 2350.4, 1810.2]], "durations": [[0, 452.1, 380.7]] }
```

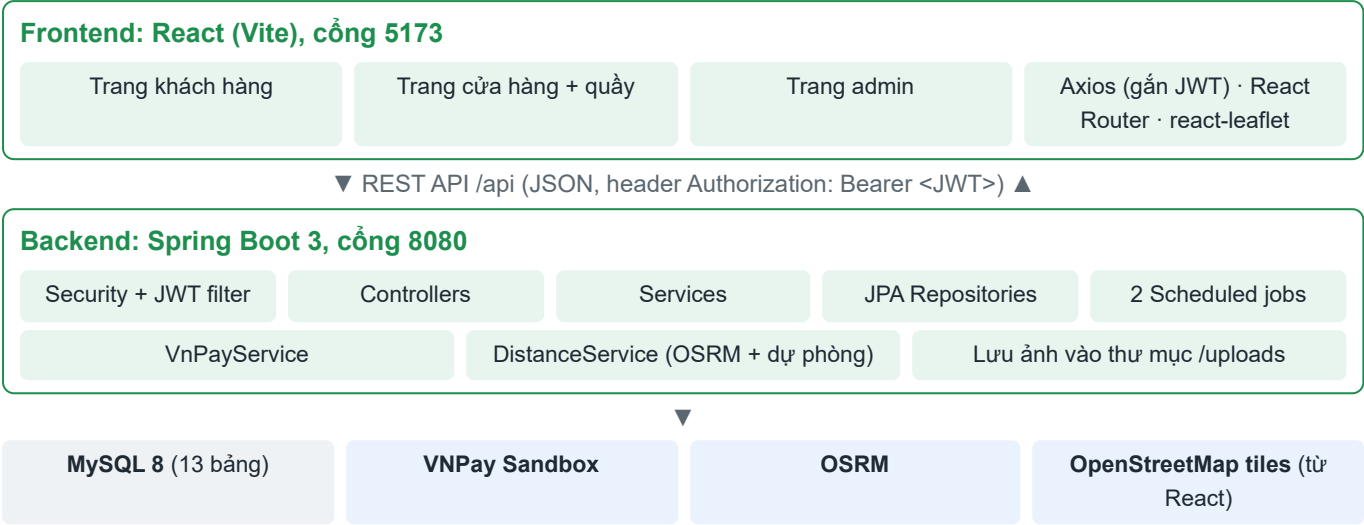
Server OSRM công khai chỉ dành cho mức dùng nhẹ và chỉ có profile ô tô; đủ cho đồ án. Muốn tính cho xe máy thì có thể thay bằng Google Distance Matrix (cần key).

b) Phía React

- `navigator.geolocation.getCurrentPosition` lấy vị trí khách; lỗi thì dùng trung tâm TP.HCM và cho bấm chọn trên bản đồ.
- `react-leaflet`: tile `https://tile.openstreetmap.org/{z}/{x}/{y}.png`, ghi "© OpenStreetMap contributors"; marker cửa hàng, popup xem nhanh túi.
- Form đăng ký cửa hàng: bản đồ cho chủ quán bấm để ghim vị trí, lấy `latitude / longitude`.

7. Kiến trúc và công nghệ

7.1 Kiến trúc



7.2 Công nghệ

Tầng	Thư viện
Backend	Java 17, Spring Boot 3 (Web, Data JPA, Security, Validation), jjwt, Lombok, MySQL Connector/J, Maven. Tuỳ chọn: springdoc-openapi (Swagger).
Frontend	React 18 + Vite, React Router, Axios, Ant Design (giao diện, bảng, form), react-leaflet, qrcode.react, Recharts (biểu đồ doanh thu), dayjs.
CSDL	MySQL 8 (MySQL Workbench hoặc XAMPP/phpMyAdmin để quản lý).
Công cụ	GitHub, Postman, Trello, Figma (tuỳ chọn).

7.3 Đăng nhập với JWT (đơn giản)

- POST /api/auth/login trả về JWT (hạn 24 giờ) chứa `userId`, `role`, `storeId`. React lưu vào `localStorage`; Axios tự gắn header `Authorization`.
- Backend: filter đọc JWT → đặt người dùng vào `SecurityContext`; các API dùng `@PreAuthorize("hasRole('ADMIN')")` ... Với chủ/nhân viên, service luôn lọc theo `storeId` lấy từ token (không tin `storeId` gửi lên).
- Gặp lỗi 401 thì React xóa token và chuyển về trang đăng nhập. CORS cho phép `http://localhost:5173`.

7.4 Chống bán vượt số lượng

Hai người cùng đặt túi cuối cùng: nếu đọc số còn lại rồi mới trừ trong Java thì cả hai đều thấy còn 1. Cách xử lý là trừ bằng một câu UPDATE có điều kiện, MySQL đảm bảo chỉ một người thành công:

```
@Modifying
@Query("""
    UPDATE RescueBatch b
    SET b.remainingQuantity = b.remainingQuantity - :qty,
        b.status = CASE WHEN b.remainingQuantity - :qty = 0 THEN 'SOLD_OUT' ELSE b.status END
    WHERE b.id = :id AND b.status = 'AVAILABLE' AND b.remainingQuantity >= :qty
""")
int decrementStock(Long id, int qty); // trả về 0 → hết hàng → báo lỗi "Túi vừa được giải cứu hết"
```

CSDL có thêm ràng buộc `CHECK (remaining_quantity >= 0)` để chặn lần cuối.

Đơn nhiều túi: gọi `decrementStock` lần lượt cho từng túi **theo thứ tự id tăng dần** (hai đơn cùng lúc khoá túi theo cùng thứ tự nên không deadlock) trong cùng một `@Transactional`. Một túi trả về 0 thì ném lỗi, transaction rollback, các túi đã trừ trước đó được trả lại tự động. Mỗi lần trừ ghi 1 dòng sổ kho `ORDERED` trong cùng transaction.

7.5 Cấu trúc mã nguồn

```
backend/src/main/java/com/foodrescue
├── config/      SecurityConfig, CorsConfig,
│               WebConfig (phục vụ /uploads)
├── security/    JwtService, JwtAuthFilter
├── entity/      User, Store, Category, RescueBatch,
│               CartItem, Order, OrderItem, Payment,
│               Refund, Review, Payout, StockMovement,
│               AuditLog
├── repository/
├── service/     AuthService, StoreService,
│               BatchService, OrderService,
│               PaymentService, VnPayService,
│               DistanceService, ReviewService,
│               ReportService, PayoutService,
│               RefundService, AuditService,
│               ReconciliationService, CartService,
│               StockService
├── scheduler/   OrderJobs
├── controller/  AuthController, PublicController,
│               CustomerController, StoreController,
│               AdminController, PaymentController
├── dto/
├── exception/   GlobalExceptionHandler
resources/application.yml
```

```
frontend/src
├── api/         axios.js, auth.js, batches.js,
│               orders.js, store.js, admin.js
├── components/  StoreMap, LocationPicker,
│               BatchCard, OrderStatusTag, ...
├── layouts/     CustomerLayout, StoreLayout,
│               AdminLayout
├── pages/
│   ├── customer/ Home, BatchDetail, NearbyMap, Cart,
│   │               PaymentResult, MyOrders, Profile
│   ├── auth/     Login, Register, StoreRegister
│   ├── store/    Counter, Batches, Orders,
│   │               Staff, Revenue, StoreProfile,
│   │               StockLog
│   └── admin/    Dashboard, Stores, Users,
│               Categories, Orders, Payouts
├── routes/      AppRoutes, PrivateRoute (theo role)
└── utils/       money.js, distance.js
```

7.6 Chạy dự án trên máy

1. Chạy `foodrescue_schema.sql` trong MySQL.
2. Backend: sửa `application.yml` (tài khoản MySQL, `VNPAY_TMN_CODE`, `VNPAY_HASH_SECRET`, `JWT_SECRET`) → `mvn spring-boot:run`.
3. Frontend: `npm install` → `npm run dev` → mở `http://localhost:5173`.
4. Deploy để demo online (tuỳ chọn): frontend lên Vercel, backend lên Render/Railway, MySQL trên Railway/Aiven.

Cấu hình bắt buộc để thời gian và tiền đúng (BR-26):

```
# application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/foodrescuedb?serverTimezone=Asia/Ho_Chi_Minh&characterEncoding=UTF-8
  jpa:
    hibernate.ddl-auto: validate          # bảng tạo từ foodrescue_schema.sql, không để Hibernate tự tạo
    properties.hibernate.jdbc.time_zone: Asia/Ho_Chi_Minh
  jackson:
    time-zone: Asia/Ho_Chi_Minh
    serialization.write-dates-as-timestamps: false # trả "2026-09-25T21:00:00"

# Chạy JVM: java -Duser.timezone=Asia/Ho_Chi_Minh -jar app.jar
# (hoặc đặt TimeZone.setDefault(TimeZone.getTimeZone("Asia/Ho_Chi_Minh")) trong main)
```

Script CSDL cần **MySQL 8.0.16 trở lên** (ràng buộc CHECK và trigger). Script đã chạy thử trên MySQL 8.0.40 (13 bảng, 25 ràng buộc CHECK, 4 trigger). XAMPP cài sẵn MariaDB 10.4: vẫn chạy được, nhưng nên dùng MySQL 8 cho giống môi trường nộp bài.

8. Danh sách API

Quyền: **P** công khai · **C** khách hàng · **S** nhân viên · **O** chủ cửa hàng · **A** admin.

Method	Endpoint	Mô tả	Quyền
Xác thực			
POST	/api/auth/register	Đăng ký khách hàng	P
POST	/api/auth/register-store	Đăng ký chủ cửa hàng + cửa hàng (multipart, có ảnh giấy phép)	P
POST	/api/auth/login	Đăng nhập, trả JWT	P
GET/PUT	/api/me , /api/me/password	Hồ sơ, đổi mật khẩu	Đăng nhập
Công khai			
GET	/api/categories	Danh mục	P
GET	/api/batches?lat&lng&radiusKm&categoryId&sort	Túi gần vị trí, kèm khoảng cách, thời gian	P
GET	/api/batches/{id}?lat&lng	Chi tiết túi	P
GET	/api/stores/{id} , /api/stores/{id}/reviews	Thông tin cửa hàng, đánh giá	P
Khách hàng			
GET	/api/cart	Giỏ hàng (giá, số còn lại mới nhất, đánh dấu túi hết)	C
POST/PUT/DELETE	/api/cart/items {batchId, quantity}, /api/cart/items/{batchId} , /api/cart	Thêm, sửa số lượng, xoá 1 túi, xoá hết giỏ	C
POST	/api/orders	Tạo đơn từ giỏ hàng, trả paymentUrl	C
GET	/api/orders , /api/orders/{id}	Đơn của tôi, chi tiết + mã nhận hàng	C
POST	/api/orders/{id}/review	Đánh giá	C
POST	/api/orders/{id}/cancel	Khách huỷ đơn chưa thanh toán	C
GET	/api/payments/vnpay/return?{vnp_*}	Xác nhận kết quả từ trang kết quả	C
GET	/api/payments/vnpay/ipn	VNPay gọi (khi deploy)	VNPay
Cửa hàng			
GET	/api/store/orders?date&status	Đơn của cửa hàng	S, O
GET	/api/store/pickup/{code}	Tra đơn theo mã nhận hàng	S, O
POST	/api/store/pickup/{code}/complete	Xác nhận đã giao	S, O
PATCH	/api/store/batches/{id}/quantity {delta}	Thêm (+) / bớt (-) túi	S, O
GET	/api/store/batches/{id}/stock-movements	Sổ kho của túi	S, O
GET/POST/PUT	/api/store/batches , /api/store/batches/{id}	Danh sách, tạo, sửa túi (multipart có ảnh)	O
POST	/api/store/batches/{id}/cancel	Huỷ túi (huỷ luôn các đơn đã trả tiền, chuyển chờ hoàn tiền)	O
POST	/api/store/orders/{id}/cancel	Huỷ đơn (lý do)	O

Method	Endpoint	Mô tả	Quyền
GET/PUT	/api/store/profile	Hồ sơ cửa hàng	O
GET/POST	/api/store/staff , /api/store/staff/{id}/lock unlock	Nhân viên	O
GET	/api/store/revenue?from&to	Doanh thu tổng hợp, biểu đồ	O
GET	/api/store/statement?from&to	Sao kê từng đơn (thực nhận, kỳ chi trả)	O
GET	/api/store/payouts , /api/store/payouts/{id}	Các kỳ đã chi, chi tiết đơn trong kỳ	O
POST	/api/store/payouts/request	Yêu cầu rút tiền (tạo kỳ PENDING)	O
Admin			
GET	/api/admin/dashboard?from&to	Thống kê	A
GET/POST	/api/admin/stores?status , /api/admin/stores/{id}/approve reject lock unlock	Duyệt (kèm tỉ lệ phí), khoá cửa hàng	A
PUT	/api/admin/stores/{id}/commission {rate}	Đổi tỉ lệ phí (áp dụng cho đơn mới)	A
GET/POST	/api/admin/users , /api/admin/users/{id}/lock unlock	Người dùng	A
POST	/api/admin/users/{id}/reset-password	Đặt lại mật khẩu tạm (trả về cho admin 1 lần)	A
CRUD	/api/admin/categories	Danh mục	A
GET/POST	/api/admin/orders? status&paymentStatus&code , /api/admin/orders/{id} , /api/admin/orders/{id}/cancel	Đơn hàng + lịch sử tiền, huỷ đơn	A
GET/POST	/api/admin/refunds?status , /api/admin/refunds/{id}/complete {gatewayRef, note}	Danh sách hoàn tiền, xác nhận đã hoàn	A
GET	/api/admin/payouts/unpaid-summary	Tiền chưa chi của từng cửa hàng	A
GET	/api/admin/payouts/preview?storeId&from&to	Xem trước đơn đủ điều kiện và tổng tiền	A
GET/POST	/api/admin/payouts {storeId, from, to}, /api/admin/payouts/{id}	Tạo kỳ chi trả; danh sách, chi tiết kỳ	A
POST	/api/admin/payouts/{id}/paid {transferRef}, /api/admin/payouts/{id}/cancel	Xác nhận đã chuyển khoản; huỷ kỳ PENDING	A
GET	/api/admin/finance/reconciliation?from&to	Đối soát 7 khoản tiền + chênh lệch	A
GET	/api/admin/audit-logs? action&entityType&entityId&from&to	Nhật ký thao tác	A

Lỗi trả về thống nhất dạng `ApiError: { "code": "BATCH_SOLD_OUT", "message": "Túi vừa được giải cứu hết", "details": [], "timestamp": "...", "path": "..." }` với HTTP 400, 401, 403, 404, 409, 422. Danh sách mã lỗi, quy ước đặt tên và giao diện: xem tài liệu **FoodRescue_QuyUocChung.pdf**.

9. Danh sách màn hình

Khách hàng

- / Trang chủ, danh sách túi (lọc, sắp xếp)
- /map Bản đồ quanh tôi
- /batches/:id Chi tiết túi, thêm vào giỏ
- /cart Giỏ hàng, đặt hàng
- /payment/result Kết quả thanh toán
- /orders Đơn của tôi (mã, QR, chỉ đường, đánh giá)
- /login · /register · /profile

Nhân viên

- /store/counter Màn hình quầy: đơn hôm nay + ô nhập mã giao hàng
- /store/batches Thêm / bớt túi, xem sổ kho

Chủ cửa hàng

- /store/register Đăng ký cửa hàng (ghim bản đồ)
- /store/counter Màn hình quầy
- /store/batches Quản lý túi (form tạo/sửa)
- /store/orders Đơn hàng, hủy đơn
- /store/staff Nhân viên
- /store/revenue Doanh thu, sao kê từng đơn, kỳ chi trả, đánh giá
- /store/profile Hồ sơ cửa hàng

Admin

- /admin Thống kê
- /admin/stores Duyệt / khoá cửa hàng
- /admin/users Người dùng
- /admin/categories Danh mục
- /admin/orders Đơn hàng + lịch sử tiền
- /admin/refunds Hoàn tiền
- /admin/payouts Chi trả theo kỳ
- /admin/finance Đối soát
- /admin/audit-logs Nhật ký thao tác

Tổng cộng khoảng 27 màn hình.

10. Thiết kế cơ sở dữ liệu (MySQL 8)

Gồm **13 bảng**. Vai trò lưu trữ trong cột `users.role`; nhân viên gắn với cửa hàng qua `users.store_id`. Tiền dùng `DECIMAL(12,0)` (VNĐ không có số lẻ). Cột in đậm là khoá chính, cột màu cam là khoá ngoại.

users id store_id (nhân viên) role, email (UQ), password_hash, full_name, phone, status, must_change_password	stores id owner_id (UQ) name, phone, address, latitude, longitude, image_url, license_image_url, open/close_time, status, reject_reason, commission_rate, bank_* (bắt buộc)	categories id name (UQ), image_url, sort_order	rescue_batches id store_id, category_id title, description, image_url, original_price, rescue_price, quantity, remaining_quantity, pickup_start, pickup_end, status
cart_items customer_id, batch_id quantity (1-5), added_at	orders id customer_id, store_id, payout_id, cancelled_by, handled_by order_code (UQ), item_count, total_amount, commission_rate, commission_amount, store_earning, pickup_start, pickup_end, pickup_code (UQ), status, payment_status, cancel_reason, cancelled_at, payment_expires_at, completed_at	order_items id order_id, batch_id (UQ cặp) quantity, unit_price, subtotal	stock_movements id batch_id, order_id, actor_id change_qty (±), reason, remaining_after, note, created_at
payments id order_id amount, txn_ref (UQ), gateway_txn_no, bank_code, response_code, status, paid_at	reviews id order_id (UQ), customer_id, store_id rating (1-5), comment	payouts id store_id, created_by, paid_by payout_code (UQ), period_from, period_to, order_count, gross_amount, commission_amount, net_amount, bank_* (chụp lại), status, transfer_ref, paid_at	refunds id order_id (UQ), payment_id, requested_by, processed_by refund_code (UQ), amount, reason, status, gateway_ref, requested_at, processed_at
audit_logs id actor_id (NULL = hệ thống) action, entity_type, entity_id, amount, detail, created_at			

Quan hệ

Quan hệ	Loại	Ghi chú
users (chủ) – stores	1 – 1	Mỗi chủ đúng 1 cửa hàng (owner_id UNIQUE).
stores – users (nhân viên)	1 – N	Qua cột users.store_id.
stores – rescue_batches	1 – N	Túi thuộc cửa hàng.
orders – order_items – rescue_batches	N – N	1 đơn nhiều túi, 1 túi nằm trong nhiều đơn; orders.store_id là cửa hàng chung của mọi túi trong đơn.
users – cart_items – rescue_batches	N – N	Giỏ hàng của khách.
rescue_batches – stock_movements	1 – N	Sổ kho của từng túi.

Quan hệ	Loại	Ghi chú
orders – payments	1 – N	Mỗi lần thanh toán là một giao dịch VNPay.
orders – reviews	1 – 0..1	Mỗi đơn tối đa 1 đánh giá.
stores – payouts	1 – N	Các kỳ chi trả của cửa hàng.
payouts – orders	1 – N	Qua <code>orders.payout_id</code> : kỳ chi trả gồm những đơn nào.
orders – refunds	1 – 0..1	Mỗi đơn hoàn tối đa 1 lần; refund trở về giao dịch VNPay gốc (<code>payment_id</code>).
users – audit_logs	1 – N	Người thực hiện thao tác.

Ràng buộc chính

- `chk_batch_price`: $0 < \text{rescue_price} \leq \text{original_price} \times 0.6$ · `chk_batch_qty`: $0 \leq \text{remaining_quantity} \leq \text{quantity}$ · `chk_batch_time`: `pickup_end > pickup_start`.
- `chk_items_money`: `subtotal = unit_price × quantity` · `chk_items_qty`, `chk_cart_qty`: 1-5 cái mỗi loại · `chk_order_pickup`: khung giờ hợp lệ.
- `chk_stock_sign`: LISTED/ADDED/RETURNED phải dương, REMOVED/ORDERED phải âm · `chk_stock_order`: ORDERED/RETURNED bắt buộc có đơn.
- `chk_users_role`: chỉ nhận 4 vai trò; `chk_review_rating`: 1-5.
- `chk_order_money` kiểm tra cả `commission_amount = ROUND(total × rate / 100)`; `chk_order_payout`: chỉ đơn COMPLETED/NO_SHOW mới có `payout_id`.
- `chk_users_staff`: nhân viên bắt buộc có `store_id`; `chk_store_rate`: phí 0-50%; `chk_order_done_time`: COMPLETED/NO_SHOW phải có `completed_at`.
- `chk_payout_money`: `net = gross - commission`; `chk_payout_paid`: PAID phải có mã chuyển khoản; `chk_refund_done`: COMPLETED phải có mã giao dịch hoàn.
- UNIQUE: `users.email`, `orders.order_code`, `orders.pickup_code`, `payments.txn_ref`, `reviews.order_id`, `refunds.order_id`, `payouts.payout_code`, `order_items(order_id, batch_id)`.

Ghi chú khi viết entity JPA

Vấn đề	Cách làm
Khoá chính ghép <code>cart_items</code>	Dùng <code>@IdClass(CartItemId.class)</code> (2 trường <code>customerId</code> , <code>batchId</code>) hoặc <code>@EmbeddedId</code> ; lớp id phải có <code>equals / hashCode</code> .
<code>users</code> ⇔ <code>stores</code> tham chiếu lẫn nhau	Lưu theo thứ tự chủ → cửa hàng → nhân viên . Trong entity: <code>Store.owner</code> là <code>@OneToOne</code> , <code>User.store</code> là <code>@ManyToOne</code> (chỉ nhân viên có). Không dùng cascade giữa hai bên.
Enum trạng thái	<code>@Enumerated(EnumType.STRING)</code> , tên hằng trùng đúng chuỗi trong SQL (<code>PENDING_PAYMENT</code> , <code>REFUND_PENDING</code> ...). Không dùng <code>ORDINAL</code> .
Tiền	<code>BigDecimal</code> (không dùng <code>double</code>); làm tròn phí bằng <code>setScale(0, RoundingMode.HALF_UP)</code> cho khớp <code>ROUND()</code> của MySQL.
Thời gian	<code>LocalDateTime</code> cho cột <code>DATETIME</code> , <code>LocalDate</code> cho <code>DATE</code> , <code>LocalTime</code> cho <code>TIME</code> . Múi giờ cấu hình ở mục 7.6.
Bảng chỉ thêm	<code>StockMovement</code> , <code>AuditLog</code> : đặt <code>@Immutable</code> (Hibernate) và không viết hàm update/delete. CSDL có trigger chặn nếu lỡ gọi.
Tạo bảng	<code>spring.jpa.hibernate.ddl-auto=validate</code> (hoặc <code>none</code>): bảng tạo từ <code>foodrescue_schema.sql</code> , không để Hibernate tự tạo (sẽ mất các CHECK, trigger, index).

Vấn đề	Cách làm
Quan hệ lazy	Mọi @ManyToOne đặt fetch = FetchType.LAZY ; API trả DTO, không trả thẳng entity (tránh vòng lặp JSON và lộ password_hash).

11. Kế hoạch thực hiện (8 tuần)

11.1 Phân công (nhóm 4 người, chia theo module)

Mỗi người làm cả backend và frontend cho module của mình để làm song song, không phải chờ nhau. Chi tiết từng việc, giờ ước tính và quy trình Git ở tài liệu **FoodRescue_KeHoachPhanCong.pdf**.

Thành viên	Phụ trách (BE + FE)
TV1: Nền tảng, Tài khoản, Admin (nhóm trưởng)	Khung dự án, entity, Security JWT, sổ kho / nhật ký / hoàn tiền dùng chung; đăng ký/đăng nhập/hồ sơ; admin đơn hàng + hoàn tiền, chi trả theo kỳ, đối soát, trang nhật ký; gộp code, deploy, tổng hợp báo cáo.
TV2: Cửa hàng và Túi	Đăng ký cửa hàng + ghim bản đồ; admin duyệt cửa hàng; CRUD túi, huỷ túi, thêm / bớt túi, sổ kho; hồ sơ cửa hàng; tài khoản nhân viên; doanh thu + sao kê; admin quản lý người dùng.
TV3: Khách hàng, Bản đồ, Đánh giá	Trang chủ, lọc, sắp xếp; tính khoảng cách OSRM ; bản đồ, định vị; chi tiết túi; trang cửa hàng; đánh giá; dữ liệu demo; admin danh mục, thống kê chung.
TV4: Đơn hàng, VNPay, Giao hàng	Giỏ hàng, tạo đơn nhiều túi + chống bán vượt; VNPay Sandbox ; đơn của tôi + mã/QR; màn hình quầy, giao hàng bằng mã; đơn của cửa hàng, huỷ đơn; 2 tác vụ tự động.

11.2 Tiến độ

Giai đoạn	Kết quả cần đạt
Tuần 1	TV1 dựng khung dự án trong 2 ngày đầu; các thành viên khác thử VNPay, OSRM và làm giao diện tĩnh; chốt API + JSON mẫu.
Sprint 1 Tuần 2-3	Đăng ký/đăng nhập JWT 4 vai trò; đăng ký cửa hàng + ghim bản đồ + admin duyệt; CRUD túi + ảnh; danh mục; danh sách túi công khai.
Sprint 2 Tuần 4-5	Tìm theo vị trí + khoảng cách OSRM + bản đồ; giỏ hàng + đặt hàng; thanh toán VNPay; đơn của tôi + mã/QR; màn hình quầy + giao hàng bằng mã; 2 tác vụ tự động. Mốc: chạy trọn luồng mua hàng.
Sprint 3 Tuần 6-7	Huỷ đơn + hoàn tiền; đánh giá; nhân viên; doanh thu + sao kê; admin người dùng, đơn hàng, chi trả theo kỳ, đối soát, nhật ký, thống kê.
Tuần 8	Test theo mục 13, sửa lỗi, làm đẹp giao diện, nhập dữ liệu mẫu, (tùy chọn) deploy, báo cáo và slide, tập demo.

Hạng mục	1	2	3	4	5	6	7	8
Thiết kế, khởi tạo								
Đăng nhập, cửa hàng, túi								
Tìm kiếm, bản đồ, OSRM								
Đặt hàng, VNPay, giao hàng								
Hoàn tiền, đánh giá, nhân viên								
Doanh thu, admin								
Kiểm thử, báo cáo								

12. Rủi ro

Rủi ro	Cách phòng tránh
Tích hợp VNPay lỗi (sai chữ ký, lệch giờ)	Làm thử ngay tuần 1; so sánh với code mẫu Java của VNPay; xác nhận qua Return URL nên không cần IPN khi chạy local.
OSRM chậm hoặc không truy cập được	Timeout 2 giây và dự phòng khoảng cách ước tính; website vẫn chạy.
Bán vượt số lượng	UPDATE có điều kiện + CHECK trong CSDL; test bằng 2 trình duyệt đặt cùng lúc.
Frontend và backend lệch API	Chốt danh sách API (mục 8) từ tuần 1; test bằng Postman trước khi nối giao diện.
Trễ tiến độ	Ưu tiên luồng mua hàng (Sprint 2); phần Sprint 3 có thể đơn giản hoá thêm nếu thiếu thời gian.
Demo lỗi mạng	Chạy local, chuẩn bị sẵn dữ liệu mẫu, quay video demo dự phòng.

13. Kiểm thử

Mã	Kịch bản	Kết quả mong đợi
TC01	2 trình duyệt cùng đặt túi chỉ còn 1	1 người thành công, người kia báo "Túi vừa được giải cứu hết"; số còn lại = 0, túi SOLD_OUT.
TC02	Thanh toán VNPAY bằng thẻ test NCB	Quay về trang kết quả, đơn CONFIRMED + PAID, hiện mã nhận hàng.
TC03	Bấm "Huỷ" trên trang VNPAY	Đơn EXPIRED, số lượng được trả lại.
TC04	Sửa tham số vnp_Amount trên URL trả về	Báo sai chữ ký, đơn không đổi.
TC05	Đặt hàng rồi không thanh toán quá 15 phút	Đơn EXPIRED, số lượng được trả lại.
TC06	Tìm túi quanh vị trí; đổi URL OSRM sang địa chỉ sai	Vẫn ra danh sách, khoảng cách hiện dấu "~".
TC07	Nhân viên cửa hàng A nhập mã đơn của cửa hàng B	Báo "Không tìm thấy đơn", không đổi trạng thái.
TC08	Nhập mã giao hàng trước giờ lấy 1 tiếng	Báo "Chưa tới giờ nhận hàng".
TC09	Tạo túi giá 70.000 / giá gốc 100.000	Báo lỗi "Giá giải cứu phải giảm ít nhất 40%".
TC10	Chủ cửa hàng huỷ đơn đã thanh toán; admin xác nhận hoàn (có và không nhập mã giao dịch)	Đơn CANCELLED + REFUND_PENDING, số lượng trả lại, có 1 bản ghi refunds 45.000đ; không nhập mã thì bị chặn; nhập mã thì refunds COMPLETED, đơn REFUNDED; nhật ký có REFUND_REQUESTED và REFUND_COMPLETED.
TC11	Khách gọi API admin / API cửa hàng	403 Forbidden.
TC12	Đánh giá đơn chưa nhận hoặc đánh giá lần 2	Bị từ chối.
TC13	Tạo kỳ chi trả cho Cơm Tấm 365, kỳ 21-25/09	Xem trước 1 đơn, gross 22.000, phí 3.300, net 18.700; tạo kỳ PENDING; nhập mã chuyển khoản → PAID; chủ cửa hàng thấy kỳ trong sao kê.
TC14	Tạo lại kỳ chi trả trùng khoảng ngày đã chi	Không có đơn đủ điều kiện, không tạo được kỳ (không chi trùng).
TC15	Admin cố huỷ một đơn COMPLETED	Bị chặn (BR-13).
TC16	Mở trang đối soát sau các thao tác trên	Các khoản cộng lại bằng tiền thu, chênh lệch = 0.
TC17	Giả lập thanh toán về sau khi đơn đã EXPIRED (gọi lại URL trả về đã lưu), lần 1 túi còn hàng, lần 2 túi hết	Lần 1: đơn CONFIRMED, số lượng bị trừ lại. Lần 2: đơn CANCELLED + REFUND_PENDING, có refunds lý do LATE_PAYMENT.
TC18	Nhân viên thêm 3 túi rồi bớt 2 túi	quantity và remaining_quantity cùng tăng 3 rồi giảm 2; bớt quá số còn lại bị chặn; C1 vẫn trả về 0 dòng.
TC19	Chủ Cơm Tấm 365 bấm "Yêu cầu rút tiền" 2 lần	Lần 1 tạo kỳ PENDING 18.700đ; lần 2 bị chặn vì đang có kỳ PENDING (BR-20).
TC20	Chạy foodrescue_checks.sql sau toàn bộ test	C1-C12 trả về 0 dòng, C0 chênh lệch = 0.
TC21	Giỏ đang có túi Sweet Paris, thêm túi của An Phát	API trả lỗi "giỏ khác cửa hàng"; React hỏi xoá giỏ cũ; đồng ý thì giỏ chỉ còn túi An Phát.
TC22	Đặt đơn 2 loại túi, trong đó 1 loại vừa hết	Báo đúng túi bị thiếu; không túi nào bị trừ; không tạo đơn; giỏ vẫn còn nguyên.

Mã	Kịch bản	Kết quả mong đợi
TC23	Đăng túi 10 cái, thêm 2, bớt 1, khách đặt 3, đơn hết hạn	Sổ kho có 5 dòng LISTED +10, ADDED +2, REMOVED -1, ORDERED -3, RETURNED +3; còn lại 11, quantity 11; C1b sạch.
TC24	Admin đặt lại mật khẩu cho khách, khách đăng nhập bằng mật khẩu tạm	Bị chuyển tới trang đổi mật khẩu; gọi API khác trả 403 cho đến khi đổi; nhật ký có USER_PASSWORD_RESET.
TC25	Chạy UPDATE hoặc DELETE trên stock_movements / audit_logs	MySQL báo lỗi (trigger), dữ liệu không đổi.
TC26	Chủ cửa hàng huỷ túi đang có 1 đơn đã trả tiền và 1 đơn chưa trả tiền	Đơn đã trả: CANCELLED + REFUND_PENDING, có refunds BATCH_CANCELLED. Đơn chưa trả: CANCELLED + UNPAID. Cả hai trả kho (RETURNED); C1-C12 vẫn sạch.

14. Tài khoản và kịch bản demo

14.1 Tài khoản mẫu (mật khẩu: 123456)

Email	Vai trò	Tình trạng dữ liệu
admin@foodrescue.vn	Admin	1 cửa hàng chờ duyệt (GreenMart); 1 yêu cầu hoàn tiền 45.000đ đang chờ; An Phát đã được chi 170.000đ (kỳ PO-1-20260924); Cơm Tấm 365 còn 18.700đ chưa chi.
owner.anphat@gmail.com	Chủ cửa hàng	Có 2 đơn đã giao, đánh giá 5 và 4 sao, đã được chi 170.000đ (có mã chuyển khoản).
owner.sweetparis@gmail.com	Chủ cửa hàng	Có đơn đang chờ lấy, đơn đã huỷ, đơn chờ thanh toán.
owner.comtam365@gmail.com	Chủ cửa hàng	Có đơn khách không đến lấy (NO_SHOW), 18.700đ chưa được chi, dùng để demo yêu cầu rút tiền.
owner.greenmart@gmail.com	Chủ cửa hàng	Cửa hàng đang chờ duyệt.
staff.anphat@gmail.com	Nhân viên	Nhân viên An Phát.
staff.sweetparis@gmail.com	Nhân viên	Nhân viên Sweet Paris.
an.malyhoang@gmail.com	Khách hàng	Có đơn 2 loại túi (87.000đ) đang chờ lấy tại Sweet Paris, mã H2R9W7.
linh.tranhoang@gmail.com	Khách hàng	Giỏ hàng đang có 2 loại túi của Sweet Paris, dùng để demo đặt hàng từ giỏ.

14.2 Kịch bản demo (khoảng 8 phút)

- Admin** duyệt cửa hàng GreenMart (xem giấy phép và vị trí trên bản đồ).
- Chủ GreenMart** đăng "Túi rau củ cuối ngày"; thử nhập giá chỉ giảm 30% để thấy bị chặn.
- Khách** mở bản đồ quanh tôi: thấy các túi kèm "x km · y phút"; thêm 2 loại túi của cùng quán vào giỏ, đặt hàng và thanh toán VNPay bằng thẻ test.
- Nhân viên** mở màn hình quầy, thấy đơn mới, nhập mã khách đọc và bấm "Đã giao".
- Khách** đánh giá 5 sao.
- Hai trình duyệt cùng đặt túi còn 1 để chứng minh không bán vượt.
- Chủ Cơm Tấm 365** xem doanh thu và sao kê từng đơn, thêm 3 túi cho túi đang bán, bấm "Yêu cầu rút tiền"; tạo tài khoản nhân viên.

8. **Admin** xác nhận hoàn tiền đơn bị huỷ (nhập mã giao dịch hoàn), xác nhận yêu cầu rút tiền của Cơm Tấm 365 bằng mã chuyển khoản, mở trang đối soát thấy chênh lệch = 0, xem nhật ký thao tác.

Phụ lục A. Script cơ sở dữ liệu (MySQL 8)

File đi kèm: `foodrescue_schema.sql` (13 bảng, 25 ràng buộc CHECK, 4 trigger + dữ liệu mẫu; đã chạy thử trên MySQL 8.0.40).
File kiểm tra dữ liệu: `foodrescue_checks.sql` (mục 5.10). Lưu ý: script bắt đầu bằng `DROP DATABASE IF EXISTS foodrescuedb`.

```
-- =====
-- FOODRESCUE - Hệ thống giải cứu thực phẩm cuối ngày (bản rút gọn)
-- Script CSDL v5.4 - MySQL 8.0.16+ (utf8mb4) - 13 bảng
-- v5 : quản lý tiền rõ ràng (refunds, payouts theo kỳ gán với đơn, tỉ lệ phí lưu theo đơn),
--      nhật ký thao tác (audit_logs)
-- v5.1: tỉ lệ phí theo cửa hàng, cửa hàng tự yêu cầu rút tiền, 1 chủ = 1 cửa hàng,
--        hoàn tiền khi thanh toán về muộn, ràng buộc nhân viên - cửa hàng
-- v5.2: giỏ hàng (cart_items), đơn nhiều túi (order_items), sổ kho (stock_movements)
-- v5.3: múi giờ +07:00, admin đặt lại mật khẩu tạm (must_change_password),
--        trigger chặn sửa/xoá sổ kho và nhật ký
-- v5.4: danh mục có ảnh và thứ tự hiển thị (thanh danh mục trang chủ)
-- Múi giờ: mọi cột DATETIME là giờ Việt Nam. Backend phải đặt cùng múi giờ
--        (JDBC serverTimezone=Asia/Ho_Chi_Minh, JVM -Duser.timezone=Asia/Ho_Chi_Minh).
-- Mật khẩu mặc định của mọi tài khoản mẫu: 123456 (BCrypt cost 10)
-- CẢNH BÁO: script xoá và tạo lại database foodrescuedb
-- =====
DROP DATABASE IF EXISTS foodrescuedb;
CREATE DATABASE foodrescuedb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE foodrescuedb;
SET time_zone = '+07:00'; -- NOW() trong phiên này theo giờ Việt Nam

-- -----
-- 1. Người dùng (4 vai trò)
-- -----
CREATE TABLE users (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  role        VARCHAR(20) NOT NULL,          -- ADMIN, STORE_OWNER, STORE_STAFF, CUSTOMER
  email       VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  full_name   VARCHAR(100) NOT NULL,
  phone       VARCHAR(20) NOT NULL,
  store_id    BIGINT,                        -- chỉ dùng cho STORE_STAFF
  status      VARCHAR(20) NOT NULL DEFAULT 'ACTIVE', -- ACTIVE, LOCKED
  must_change_password BOOLEAN NOT NULL DEFAULT FALSE, -- TRUE: mật khẩu tạm (nhân viên mới / admin đặt lại)
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_users_role CHECK (role IN ('ADMIN', 'STORE_OWNER', 'STORE_STAFF', 'CUSTOMER')),
  -- nhân viên bắt buộc thuộc 1 cửa hàng; vai trò khác không có store_id
  CONSTRAINT chk_users_staff CHECK ((role = 'STORE_STAFF') = (store_id IS NOT NULL))
);

-- -----
-- 2. Cửa hàng
-- -----
CREATE TABLE stores (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  owner_id    BIGINT NOT NULL UNIQUE,        -- 1 chủ = 1 cửa hàng
  name        VARCHAR(150) NOT NULL,
  description  TEXT,
  phone       VARCHAR(20) NOT NULL,
  address     VARCHAR(255) NOT NULL,
  latitude    DECIMAL(10,8) NOT NULL,        -- chủ quán ghi trên bản đồ
  longitude   DECIMAL(11,8) NOT NULL,
  image_url   VARCHAR(255),
  license_image_url VARCHAR(255) NOT NULL,    -- ảnh giấy phép VSATTP
  open_time   TIME NOT NULL,
  close_time  TIME NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'PENDING', -- PENDING, APPROVED, REJECTED, LOCKED
  reject_reason VARCHAR(255),
  commission_rate DECIMAL(5,2) NOT NULL DEFAULT 15.00, -- % phí nền tảng, admin đặt; đơn mới copy sang orders
  bank_name    VARCHAR(100) NOT NULL,        -- tài khoản nhận tiền chi trả (bắt buộc)
  bank_account_no VARCHAR(30) NOT NULL,
  bank_account_name VARCHAR(100) NOT NULL,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_stores_owner FOREIGN KEY (owner_id) REFERENCES users(id),
  CONSTRAINT chk_store_time CHECK (close_time > open_time),
  CONSTRAINT chk_store_rate CHECK (commission_rate BETWEEN 0 AND 50)
);
CREATE INDEX idx_stores_status_geo ON stores(status, latitude, longitude);

ALTER TABLE users ADD CONSTRAINT fk_users_store FOREIGN KEY (store_id) REFERENCES stores(id);
```

```

-----
-- 3. Danh mục
-----
CREATE TABLE categories (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  image_url   VARCHAR(255),           -- ảnh vuông nền trong, hiện trên thanh danh mục
  sort_order  INT          NOT NULL DEFAULT 0 -- nhỏ đứng trước
);

-----
-- 4. Túi giải cứu (hàng đang bán = tồn kho hiện tại)
-----
CREATE TABLE rescue_batches (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  store_id    BIGINT          NOT NULL,
  category_id BIGINT          NOT NULL,
  title       VARCHAR(150)   NOT NULL,
  description  TEXT,
  image_url   VARCHAR(255),
  original_price DECIMAL(12,0) NOT NULL,
  rescue_price DECIMAL(12,0) NOT NULL,
  quantity    INT            NOT NULL,      -- tổng số túi đã đưa lên app = đã bán + còn lại
  remaining_quantity INT      NOT NULL,      -- còn lại để bán
  pickup_start DATETIME      NOT NULL,
  pickup_end   DATETIME      NOT NULL,
  status       VARCHAR(20)    NOT NULL DEFAULT 'AVAILABLE', -- AVAILABLE, SOLD_OUT, EXPIRED, CANCELLED
  created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_batch_store FOREIGN KEY (store_id) REFERENCES stores(id),
  CONSTRAINT fk_batch_category FOREIGN KEY (category_id) REFERENCES categories(id),
  CONSTRAINT chk_batch_price CHECK (rescue_price > 0 AND rescue_price <= original_price * 0.6),
  CONSTRAINT chk_batch_qty CHECK (quantity >= 0 AND remaining_quantity BETWEEN 0 AND quantity),
  CONSTRAINT chk_batch_time CHECK (pickup_end > pickup_start)
);
CREATE INDEX idx_batch_listing ON rescue_batches(status, pickup_end);

-----
-- 5. Kỳ chỉ trả cho cửa hàng: admin tạo theo kỳ, HOẶC chủ cửa hàng bấm "Yêu cầu rút tiền"
--   (created_by = người tạo). Gom các đơn chưa chỉ trong kỳ.
-----
CREATE TABLE payouts (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  payout_code VARCHAR(30)   NOT NULL UNIQUE, -- PO-<storeId>-yyyymmdd
  store_id    BIGINT          NOT NULL,
  period_from DATE          NOT NULL,         -- kỳ đối soát (theo ngày chốt đơn)
  period_to   DATE          NOT NULL,
  order_count INT            NOT NULL,
  gross_amount DECIMAL(14,0) NOT NULL,        -- tổng khách đã trả của các đơn trong kỳ
  commission_amount DECIMAL(14,0) NOT NULL,   -- tổng phí nền tảng
  net_amount  DECIMAL(14,0) NOT NULL,        -- số tiền chuyển cho cửa hàng
  bank_name   VARCHAR(100)  NOT NULL,         -- chụp lại tài khoản tại thời điểm tạo
  bank_account_no VARCHAR(30) NOT NULL,
  bank_account_name VARCHAR(100) NOT NULL,
  status      VARCHAR(20)   NOT NULL DEFAULT 'PENDING', -- PENDING, PAID, CANCELLED
  transfer_ref VARCHAR(100), -- mã giao dịch chuyển khoản ngân hàng
  note        VARCHAR(255),
  created_by  BIGINT          NOT NULL,        -- admin, hoặc chủ cửa hàng (yêu cầu rút tiền)
  created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  paid_by     BIGINT,
  paid_at     DATETIME,
  CONSTRAINT fk_payouts_store FOREIGN KEY (store_id) REFERENCES stores(id),
  CONSTRAINT fk_payouts_creator FOREIGN KEY (created_by) REFERENCES users(id),
  CONSTRAINT fk_payouts_payer FOREIGN KEY (paid_by) REFERENCES users(id),
  CONSTRAINT chk_payout_money CHECK (order_count > 0 AND net_amount > 0
                                     AND net_amount = gross_amount - commission_amount),
  CONSTRAINT chk_payout_period CHECK (period_to >= period_from),
  CONSTRAINT chk_payout_paid CHECK (status <> 'PAID' OR (paid_at IS NOT NULL AND transfer_ref IS NOT NULL))
);

-----
-- 6. Giỏ hàng (lưu trong CSDL; chỉ chứa túi của 1 cửa hàng - kiểm tra ở service)
--   Giỏ hàng KHÔNG giữ hàng; hàng chỉ bị trừ khi đặt đơn.
-----
CREATE TABLE cart_items (
  customer_id BIGINT NOT NULL,
  batch_id    BIGINT NOT NULL,
  quantity    INT    NOT NULL,
  added_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (customer_id, batch_id),
  CONSTRAINT fk_cart_customer FOREIGN KEY (customer_id) REFERENCES users(id),
  CONSTRAINT fk_cart_batch FOREIGN KEY (batch_id) REFERENCES rescue_batches(id),

```

```

CONSTRAINT chk_cart_qty    CHECK (quantity BETWEEN 1 AND 5)
);

-----
-- 7. Đơn hàng (1 đơn = 1 cửa hàng, nhiều túi)
-----
CREATE TABLE orders (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_code   VARCHAR(30)   NOT NULL UNIQUE,
  customer_id  BIGINT        NOT NULL,
  store_id     BIGINT        NOT NULL,
  item_count   INT           NOT NULL,      -- tổng số túi trong đơn
  total_amount DECIMAL(12,0) NOT NULL,      -- = tổng order_items.subtotal (khách trả)
  commission_rate DECIMAL(5,2) NOT NULL,    -- tỉ lệ phí của cửa hàng tại thời điểm đặt, vd 15.00
  commission_amount DECIMAL(12,0) NOT NULL,  -- = ROUND(total_amount * commission_rate / 100)
  store_earning DECIMAL(12,0) NOT NULL,      -- = total_amount - commission_amount
  pickup_start DATETIME      NOT NULL,      -- khung giờ lấy của đơn = phần giao nhau
  pickup_end   DATETIME      NOT NULL,      -- khung giờ của các túi trong đơn
  pickup_code  VARCHAR(10)   NOT NULL UNIQUE,
  status       VARCHAR(20)   NOT NULL DEFAULT 'PENDING_PAYMENT',
  -- PENDING_PAYMENT, CONFIRMED, COMPLETED, CANCELLED, EXPIRED, NO_SHOW
  -- CANCELLED + UNPAID: khách tự huỷ đơn chưa trả tiền; CANCELLED + REFUND_*: đã trả tiền rồi bị huỷ
  payment_status VARCHAR(20) NOT NULL DEFAULT 'UNPAID',
  -- UNPAID, PAID, REFUND_PENDING, REFUNDED
  payout_id      BIGINT,                  -- kỳ chi trả đã gom đơn này (NULL = chưa chi)
  cancel_reason  VARCHAR(255),
  cancelled_by   BIGINT,
  cancelled_at   DATETIME,
  handled_by     BIGINT,                  -- người giao hàng (chủ/nhân viên)
  payment_expires_at DATETIME NOT NULL,    -- link VNPAY hết hạn sau 10 phút
  completed_at   DATETIME,                -- thời điểm chốt đơn: giao hàng (COMPLETED) hoặc NO_SHOW
  created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES users(id),
  CONSTRAINT fk_orders_store    FOREIGN KEY (store_id)   REFERENCES stores(id),
  CONSTRAINT fk_orders_payout   FOREIGN KEY (payout_id)  REFERENCES payouts(id),
  CONSTRAINT fk_orders_canceller FOREIGN KEY (cancelled_by) REFERENCES users(id),
  CONSTRAINT fk_orders_handler  FOREIGN KEY (handled_by) REFERENCES users(id),
  CONSTRAINT chk_order_items    CHECK (item_count > 0),
  CONSTRAINT chk_order_money    CHECK (total_amount > 0
                                     AND commission_amount = ROUND(total_amount * commission_rate / 100)
                                     AND store_earning = total_amount - commission_amount),
  CONSTRAINT chk_order_pickup   CHECK (pickup_end > pickup_start),
  -- chỉ đơn đã hoàn tất / không đến lấy mới được đưa vào kỳ chi trả
  CONSTRAINT chk_order_payout   CHECK (payout_id IS NULL OR status IN ('COMPLETED', 'NO_SHOW')),
  CONSTRAINT chk_order_done_time CHECK ((status IN ('COMPLETED', 'NO_SHOW')) = (completed_at IS NOT NULL))
);
CREATE INDEX idx_orders_store    ON orders(store_id, status, created_at);
CREATE INDEX idx_orders_customer ON orders(customer_id, created_at);
CREATE INDEX idx_orders_payout   ON orders(store_id, payout_id, status, completed_at);

-----
-- 8. Chi tiết đơn (các túi trong đơn, giá chụp lại lúc mua)
-----
CREATE TABLE order_items (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id     BIGINT        NOT NULL,
  batch_id     BIGINT        NOT NULL,
  quantity     INT           NOT NULL,
  unit_price   DECIMAL(12,0) NOT NULL,      -- giá giải cứu tại thời điểm mua
  subtotal     DECIMAL(12,0) NOT NULL,      -- = unit_price * quantity
  CONSTRAINT fk_items_order     FOREIGN KEY (order_id) REFERENCES orders(id),
  CONSTRAINT fk_items_batch     FOREIGN KEY (batch_id) REFERENCES rescue_batches(id),
  CONSTRAINT uq_items_batch     UNIQUE (order_id, batch_id),
  CONSTRAINT chk_items_qty      CHECK (quantity BETWEEN 1 AND 5),
  CONSTRAINT chk_items_money    CHECK (subtotal = unit_price * quantity)
);

-----
-- 9. Số kho: mỗi lần số lượng túi thay đổi ghi 1 dòng (chỉ thêm, không sửa/xoá)
--   quantity = tổng change_qty của LISTED + ADDED + REMOVED
--   remaining = tổng change_qty của mọi dòng
-----
CREATE TABLE stock_movements (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  batch_id     BIGINT        NOT NULL,
  change_qty   INT           NOT NULL,      -- dương = tăng, âm = giảm
  reason       VARCHAR(20)   NOT NULL,
  -- LISTED : đăng túi (+ số lượng ban đầu)
  -- ADDED  : cửa hàng thêm túi (+)
  -- REMOVED: bớt túi, vd khách tại quầy mua mất (-)
  -- ORDERED: khách đặt đơn (-)

```

```

-- RETURNED: đơn hết hạn / huỷ, trả lại (+)
order_id      BIGINT,                -- với ORDERED / RETURNED
actor_id      BIGINT,                -- NULL = hệ thống
remaining_after INT NOT NULL,        -- số còn lại sau thay đổi
note          VARCHAR(255),
created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_stock_batch FOREIGN KEY (batch_id) REFERENCES rescue_batches(id),
CONSTRAINT fk_stock_order FOREIGN KEY (order_id) REFERENCES orders(id),
CONSTRAINT fk_stock_actor FOREIGN KEY (actor_id) REFERENCES users(id),
CONSTRAINT chk_stock_nonzero CHECK (change_qty <> 0),
CONSTRAINT chk_stock_sign CHECK (
    (reason IN ('LISTED', 'ADDED', 'RETURNED') AND change_qty > 0) OR
    (reason IN ('REMOVED', 'ORDERED') AND change_qty < 0)),
CONSTRAINT chk_stock_order CHECK ((reason IN ('ORDERED', 'RETURNED')) = (order_id IS NOT NULL)),
CONSTRAINT chk_stock_after CHECK (remaining_after >= 0)
);
CREATE INDEX idx_stock_batch ON stock_movements(batch_id, created_at);

-- Số kho chỉ được thêm, không được sửa / xoá
DELIMITER $
CREATE TRIGGER trg_stock_no_update BEFORE UPDATE ON stock_movements FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'stock_movements chỉ được thêm, không được sửa'$
CREATE TRIGGER trg_stock_no_delete BEFORE DELETE ON stock_movements FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'stock_movements chỉ được thêm, không được xoá'$
DELIMITER ;

-----
-- 10. Giao dịch thanh toán VNPay
-----
CREATE TABLE payments (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id    BIGINT NOT NULL,
    amount      DECIMAL(12,0) NOT NULL,
    txn_ref     VARCHAR(64) NOT NULL UNIQUE,    -- vnp_TxnRef
    gateway_txn_no VARCHAR(64),                -- vnp_TransactionNo (cần khi hoàn tiền)
    bank_code   VARCHAR(20),
    response_code VARCHAR(10),                -- vnp_ResponseCode
    status      VARCHAR(20) NOT NULL DEFAULT 'PENDING', -- PENDING, SUCCESS, FAILED
    paid_at     DATETIME,
    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_payments_order FOREIGN KEY (order_id) REFERENCES orders(id)
);

-----
-- 11. Hoàn tiền cho khách
-----
CREATE TABLE refunds (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    refund_code VARCHAR(30) NOT NULL UNIQUE,    -- RF-<orderCode>
    order_id    BIGINT NOT NULL UNIQUE,        -- mỗi đơn hoàn tối đa 1 lần (toàn bộ)
    payment_id  BIGINT NOT NULL,                -- giao dịch VNPay gốc
    amount      DECIMAL(12,0) NOT NULL,        -- = orders.total_amount
    reason      VARCHAR(30) NOT NULL,          -- STORE_CANCELLED, BATCH_CANCELLED, ADMIN_CANCELLED, LATE_PAYMENT
    status      VARCHAR(20) NOT NULL DEFAULT 'PENDING', -- PENDING, COMPLETED
    gateway_ref VARCHAR(100),                  -- mã giao dịch hoàn trên VNPay merchant
    note        VARCHAR(255),
    requested_by BIGINT,                        -- người huỷ đơn; NULL = hệ thống (LATE_PAYMENT)
    requested_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_by BIGINT,                        -- admin xác nhận đã hoàn
    processed_at DATETIME,
    CONSTRAINT fk_refunds_order FOREIGN KEY (order_id) REFERENCES orders(id),
    CONSTRAINT fk_refunds_payment FOREIGN KEY (payment_id) REFERENCES payments(id),
    CONSTRAINT fk_refunds_requester FOREIGN KEY (requested_by) REFERENCES users(id),
    CONSTRAINT fk_refunds_processor FOREIGN KEY (processed_by) REFERENCES users(id),
    CONSTRAINT chk_refund_amount CHECK (amount > 0),
    CONSTRAINT chk_refund_done CHECK (status <> 'COMPLETED' OR (processed_at IS NOT NULL AND gateway_ref IS NOT NULL))
);

-----
-- 12. Đánh giá (1 đơn 1 đánh giá, đánh giá cửa hàng)
-----
CREATE TABLE reviews (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_id    BIGINT NOT NULL UNIQUE,
    customer_id BIGINT NOT NULL,
    store_id    BIGINT NOT NULL,
    rating      TINYINT NOT NULL,
    comment     TEXT,
    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_reviews_order FOREIGN KEY (order_id) REFERENCES orders(id),
    CONSTRAINT fk_reviews_customer FOREIGN KEY (customer_id) REFERENCES users(id),

```

```

CONSTRAINT fk_reviews_store FOREIGN KEY (store_id) REFERENCES stores(id),
CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5)
);

-----
-- 13. Nhật ký thao tác (chỉ thêm, không sửa/xoá)
-----

CREATE TABLE audit_logs (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    actor_id BIGINT, -- NULL = hệ thống (VNPay, tác vụ tự động)
    action VARCHAR(40) NOT NULL,
    -- PAYMENT_SUCCESS, ORDER_CANCELLED, ORDER_EXPIRED, ORDER_COMPLETED, ORDER_NO_SHOW,
    -- REFUND_REQUESTED, REFUND_COMPLETED, PAYOUT_CREATED, PAYOUT_PAID, PAYOUT_CANCELLED,
    -- BATCH_CANCELLED, STORE_APPROVED, STORE_REJECTED, STORE_LOCKED, STORE_RATE_CHANGED,
    -- USER_LOCKED, USER_UNLOCKED, USER_PASSWORD_RESET
    entity_type VARCHAR(20) NOT NULL, -- ORDER, REFUND, PAYOUT, STORE, USER, BATCH
    entity_id BIGINT NOT NULL,
    amount DECIMAL(14,0), -- số tiền liên quan (nếu có)
    detail VARCHAR(500),
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_audit_actor FOREIGN KEY (actor_id) REFERENCES users(id)
);

CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_time ON audit_logs(created_at);

-- Nhật ký chỉ được thêm, không được sửa / xoá
DELIMITER $
CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON audit_logs FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_logs chỉ được thêm, không được sửa'$
CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON audit_logs FOR EACH ROW
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_logs chỉ được thêm, không được xoá'$
DELIMITER ;

-- =====
-- DỮ LIỆU MẪU ("hôm nay" = 2026-09-25)
-- =====
SET @pw = '$2a$10$Z9WS6Xe2NDqRK7M4rKoyV.uxSnXxs1lWiVAbVXxa4tvQyn1nxeNni'; -- 123456
INSERT INTO users (id, role, email, password_hash, full_name, phone, status) VALUES
( 1, 'ADMIN', 'admin@foodrescue.vn', @pw, 'Quản Trị Viên', '0909000001', 'ACTIVE'),
( 2, 'STORE_OWNER', 'owner.anphat@gmail.com', @pw, 'Nguyễn Văn Phát', '0912345671', 'ACTIVE'),
( 3, 'STORE_OWNER', 'owner.sweetparis@gmail.com', @pw, 'Lê Thị Thu Thảo', '0912345672', 'ACTIVE'),
( 4, 'STORE_OWNER', 'owner.comtam365@gmail.com', @pw, 'Trần Văn Cali', '0912345673', 'ACTIVE'),
( 5, 'STORE_OWNER', 'owner.greenmart@gmail.com', @pw, 'Võ Thị Hạnh', '0912345674', 'ACTIVE'),
( 8, 'CUSTOMER', 'an.malyhoang@gmail.com', @pw, 'Ma Lý Hoàng Ân', '0987654321', 'ACTIVE'),
( 9, 'CUSTOMER', 'linh.tranhoang@gmail.com', @pw, 'Trần Hoàng Linh', '0987654322', 'ACTIVE'),
(10, 'CUSTOMER', 'bao.lequoc@gmail.com', @pw, 'Lê Quốc Bảo', '0987654323', 'ACTIVE'),
(11, 'CUSTOMER', 'thu.phamminh@gmail.com', @pw, 'Phạm Minh Thư', '0987654324', 'ACTIVE');

INSERT INTO stores (id, owner_id, name, description, phone, address, latitude, longitude, license_image_url,
    open_time, close_time, status, commission_rate, bank_name, bank_account_no, bank_account_name) VALUES
(1, 2, 'Bánh Mì Tươi & Hamburger An Phát', 'Bánh mì, sandwich, hamburger làm trong ngày.', '0283830001',
    '105 An Dương Vương, Phường 8, Quận 5, TP.HCM', 10.75837200, 106.67123900, '/uploads/license-1.jpg',
    '06:30:00', '21:30:00', 'APPROVED', 15.00, 'Vietcombank', '0071000123456', 'NGUYEN VAN PHAT'),
(2, 3, 'Sweet Paris Bakery & Coffee', 'Bánh ngọt kiểu Pháp và cà phê.', '0283830002',
    '45 Lê Duẩn, Bến Nghé, Quận 1, TP.HCM', 10.78125400, 106.69894200, '/uploads/license-2.jpg',
    '07:00:00', '22:00:00', 'APPROVED', 15.00, 'Techcombank', '19031234567890', 'LE THI THU THAO'),
(3, 4, 'Cơm Tắm Truyền Thống Sài Gòn 365', 'Cơm tấm sườn bì chả truyền thống.', '0283830003',
    '215 Điện Biên Phủ, Phường 15, Bình Thạnh, TP.HCM', 10.79634100, 106.70921800, '/uploads/license-3.jpg',
    '10:00:00', '21:00:00', 'APPROVED', 15.00, 'ACB', '2468101214', 'TRAN VAN CALI'),
(4, 5, 'GreenMart Rau Củ Sạch', 'Rau củ quả hữu cơ.', '0283830004',
    '12 Nguyễn Thị Minh Khai, Đa Kao, Quận 1, TP.HCM', 10.78756000, 106.69980000, '/uploads/license-4.jpg',
    '07:00:00', '21:00:00', 'PENDING', 10.00, 'MB Bank', '0123456789', 'VO THI HANH');

-- Nhân viên chèn sau cửa hàng (vì bắt buộc có store_id)
INSERT INTO users (id, role, email, password_hash, full_name, phone, store_id, status) VALUES
( 6, 'STORE_STAFF', 'staff.anphat@gmail.com', @pw, 'Đỗ Minh Khoa', '0933000001', 1, 'ACTIVE'), -- cả 2 đã đổi mật khẩu
( 7, 'STORE_STAFF', 'staff.sweetparis@gmail.com', @pw, 'Huỳnh Ngọc Mai', '0933000002', 2, 'ACTIVE');

INSERT INTO categories (id, name, image_url, sort_order) VALUES
(1, 'Bánh mì & Sandwich', '/uploads/categories/banh-mi.png', 1),
(2, 'Bánh ngọt', '/uploads/categories/banh-ngot.png', 2),
(3, 'Cơm & Món chính', '/uploads/categories/com.png', 3),
(4, 'Rau củ & Trái cây', '/uploads/categories/rau-cu.png', 4),
(5, 'Đồ uống', '/uploads/categories/do-uong.png', 5),
(6, 'Tạp hoá', '/uploads/categories/tap-hoa.png', 6);

INSERT INTO rescue_batches (id, store_id, category_id, title, description, original_price, rescue_price,
    quantity, remaining_quantity, pickup_start, pickup_end, status, created_at) VALUES
(1, 1, 1, 'Túi Bánh Mì Bất Ngờ', '2 bánh mì hoa cúc mini + 1 sandwich xúc xích trong ngày.',
    80000, 30000, 10, 8, '2026-09-25 20:30:00', '2026-09-25 21:30:00', 'AVAILABLE', '2026-09-25 09:00:00'),
(2, 1, 1, 'Combo Hamburger Cuối Giờ', '2 burger bò/gà giữ nóng tại quầy.',

```

```

95000, 40000, 5, 0, '2026-09-23 20:30:00', '2026-09-23 21:30:00', 'SOLD_OUT', '2026-09-23 17:00:00'),
(3, 2, 2, 'Surprise Box Bánh Pháp', 'Ngẫu nhiên 3 loại: croissant phô mai, tart trứng, donut kem.',
120000, 45000, 15, 14, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'AVAILABLE', '2026-09-25 08:00:00'),
(4, 2, 2, 'Bánh Kem Lát Mini', 'Bánh kem bắp và tiramisu lát, bảo quản tủ mát.',
110000, 42000, 8, 6, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'AVAILABLE', '2026-09-25 08:05:00'),
(5, 3, 3, 'Suất Cơm Sườn Bì Chả', 'Cơm nóng đóng hộp gồm sườn nướng, bì, chả trứng.',
65000, 25000, 20, 18, '2026-09-25 20:00:00', '2026-09-25 21:00:00', 'AVAILABLE', '2026-09-25 10:00:00'),
(6, 3, 3, 'Cơm Tắm Cuối Ngày', 'Suất cơm tắm sườn trứng.',
60000, 22000, 10, 9, '2026-09-23 20:00:00', '2026-09-23 21:00:00', 'EXPIRED', '2026-09-23 16:00:00');

-- Kỳ chi trả: An Phát đã được chi kỳ 21-24/09 (đơn 1, 2). Cơm Tắm 365 chưa được chi (đơn 3).
INSERT INTO payouts (id, payout_code, store_id, period_from, period_to, order_count, gross_amount,
commission_amount, net_amount, bank_name, bank_account_no, bank_account_name, status,
transfer_ref, note, created_by, created_at, paid_by, paid_at) VALUES
(1, 'PO-1-20260924', 1, '2026-09-21', '2026-09-24', 2, 200000, 30000, 170000,
'Vietcombank', '0071000123456', 'NGUYEN VAN PHAT', 'PAID',
'VCB-FT26267112233', 'Chi trả tuần 21-24/09', 1, '2026-09-25 09:00:00', 1, '2026-09-25 09:30:00');

-- Đơn hàng (phí 15%). Đơn 4 gồm 2 loại túi của Sweet Paris (đặt từ giờ hàng).
INSERT INTO orders (id, order_code, customer_id, store_id, item_count, total_amount,
commission_rate, commission_amount, store_earning, pickup_start, pickup_end, pickup_code,
status, payment_status, payout_id, cancel_reason, cancelled_by, cancelled_at, handled_by,
payment_expires_at, completed_at, created_at) VALUES
(1, 'FR260923-0001', 8, 1, 2, 80000, 15.00, 12000, 68000, '2026-09-23 20:30:00', '2026-09-23 21:30:00', 'K7Q2M9', 'COMPLETED',
'PAID', 1, NULL, NULL, NULL, 6, '2026-09-23 19:25:00', '2026-09-23 20:45:00', '2026-09-23 19:15:00'),
(2, 'FR260923-0002', 9, 1, 3, 120000, 15.00, 18000, 102000, '2026-09-23 20:30:00', '2026-09-23 21:30:00', 'P3X8D1', 'COMPLETED',
'PAID', 1, NULL, NULL, NULL, 6, '2026-09-23 19:40:00', '2026-09-23 21:10:00', '2026-09-23 19:30:00'),
(3, 'FR260923-0003', 11, 3, 1, 22000, 15.00, 3300, 18700, '2026-09-23 20:00:00', '2026-09-23 21:00:00', 'T5N6B4', 'NO_SHOW',
'PAID', NULL, NULL, NULL, NULL, NULL, '2026-09-23 18:10:00', '2026-09-23 21:30:00', '2026-09-23 18:00:00'),
(4, 'FR260925-0004', 8, 2, 2, 87000, 15.00, 13050, 73950, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'H2R9W7', 'CONFIRMED',
'PAID', NULL, NULL, NULL, NULL, NULL, '2026-09-25 10:30:00', NULL, '2026-09-25 10:20:00'),
(5, 'FR260925-0005', 9, 3, 2, 50000, 15.00, 7500, 42500, '2026-09-25 20:00:00', '2026-09-25 21:00:00', 'M8C1Z5', 'CONFIRMED',
'PAID', NULL, NULL, NULL, NULL, NULL, '2026-09-25 11:55:00', NULL, '2026-09-25 11:45:00'),
(6, 'FR260925-0006', 10, 1, 2, 60000, 15.00, 9000, 51000, '2026-09-25 20:30:00', '2026-09-25 21:30:00', 'W4E7J3', 'CONFIRMED',
'PAID', NULL, NULL, NULL, NULL, NULL, '2026-09-25 12:20:00', NULL, '2026-09-25 12:10:00'),
(7, 'FR260925-0007', 11, 2, 1, 45000, 15.00, 6750, 38250, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'Y6U3S8', 'CANCELLED',
'REFUND_PENDING', NULL, 'Khách tại quầy đã mua hết', 3, '2026-09-25 13:20:00', NULL, '2026-09-25 13:10:00', NULL, '2026-09-25
13:00:00'),
(8, 'FR260925-0008', 10, 2, 1, 42000, 15.00, 6300, 35700, '2026-09-25 21:00:00', '2026-09-25 22:00:00', 'G9L2V6',
'PENDING_PAYMENT', 'UNPAID', NULL, NULL, NULL, NULL, NULL, '2026-09-25 13:40:00', NULL, '2026-09-25
13:30:00');

INSERT INTO order_items (order_id, batch_id, quantity, unit_price, subtotal) VALUES
(1, 2, 2, 40000, 80000),
(2, 2, 3, 40000, 120000),
(3, 6, 1, 22000, 22000),
(4, 3, 1, 45000, 45000),
(4, 4, 1, 42000, 42000),
(5, 5, 2, 25000, 50000),
(6, 1, 2, 30000, 60000),
(7, 3, 1, 45000, 45000),
(8, 4, 1, 42000, 42000);

-- Sổ kho (khớp với quantity / remaining_quantity ở trên)
INSERT INTO stock_movements (batch_id, change_qty, reason, order_id, actor_id, remaining_after, note, created_at) VALUES
(6, 10, 'LISTED', NULL, 4, 10, 'Đăng túi', '2026-09-23 16:00:00'),
(2, 5, 'LISTED', NULL, 2, 5, 'Đăng túi', '2026-09-23 17:00:00'),
(6, -1, 'ORDERED', 3, 11, 9, 'Đơn FR260923-0003', '2026-09-23 18:00:00'),
(2, -2, 'ORDERED', 1, 8, 3, 'Đơn FR260923-0001', '2026-09-23 19:15:00'),
(2, -3, 'ORDERED', 2, 9, 0, 'Đơn FR260923-0002', '2026-09-23 19:30:00'),
(3, 15, 'LISTED', NULL, 7, 15, 'Đăng túi', '2026-09-25 08:00:00'),
(4, 8, 'LISTED', NULL, 3, 8, 'Đăng túi', '2026-09-25 08:05:00'),
(1, 8, 'LISTED', NULL, 6, 8, 'Đăng túi', '2026-09-25 09:00:00'),
(5, 20, 'LISTED', NULL, 4, 20, 'Đăng túi', '2026-09-25 10:00:00'),
(3, -1, 'ORDERED', 4, 8, 14, 'Đơn FR260925-0004', '2026-09-25 10:20:00'),
(4, -1, 'ORDERED', 4, 8, 7, 'Đơn FR260925-0004', '2026-09-25 10:20:00'),
(1, 2, 'ADDED', NULL, 6, 10, 'Làm thêm 2 túi', '2026-09-25 11:00:00'),
(5, -2, 'ORDERED', 5, 9, 18, 'Đơn FR260925-0005', '2026-09-25 11:45:00'),
(1, -2, 'ORDERED', 6, 10, 8, 'Đơn FR260925-0006', '2026-09-25 12:10:00'),
(3, -1, 'ORDERED', 7, 11, 13, 'Đơn FR260925-0007', '2026-09-25 13:00:00'),
(3, 1, 'RETURNED', 7, 3, 14, 'Huỷ đơn FR260925-0007', '2026-09-25 13:20:00'),
(4, -1, 'ORDERED', 8, 10, 6, 'Đơn FR260925-0008', '2026-09-25 13:30:00');

-- Giờ hàng hiện tại: Linh đang chọn 2 túi của Sweet Paris; Bảo đang chọn 1 túi An Phát
INSERT INTO cart_items (customer_id, batch_id, quantity, added_at) VALUES
( 9, 3, 1, '2026-09-25 14:00:00'),
( 9, 4, 2, '2026-09-25 14:01:00'),
(10, 1, 1, '2026-09-25 14:05:00');

INSERT INTO payments (id, order_id, amount, txn_ref, gateway_txn_no, bank_code, response_code, status, paid_at) VALUES
(1, 1, 80000, 'FR260923-0001-1', '14567001', 'NCB', '00', 'SUCCESS', '2026-09-23 19:16:00'),

```

```

(2, 2, 120000, 'FR260923-0002-1', '14567002', 'NCB', '00', 'SUCCESS', '2026-09-23 19:31:00'),
(3, 3, 22000, 'FR260923-0003-1', '14567003', 'NCB', '00', 'SUCCESS', '2026-09-23 18:01:00'),
(4, 4, 87000, 'FR260925-0004-1', '14567004', 'NCB', '00', 'SUCCESS', '2026-09-25 10:21:00'),
(5, 5, 50000, 'FR260925-0005-1', '14567005', 'NCB', '00', 'SUCCESS', '2026-09-25 11:46:00'),
(6, 6, 60000, 'FR260925-0006-1', '14567006', 'NCB', '00', 'SUCCESS', '2026-09-25 12:11:00'),
(7, 7, 45000, 'FR260925-0007-1', '14567007', 'NCB', '00', 'SUCCESS', '2026-09-25 13:01:00'),
(8, 8, 42000, 'FR260925-0008-1', NULL, NULL, NULL, 'PENDING', NULL);

INSERT INTO refunds (refund_code, order_id, payment_id, amount, reason, status, requested_by, requested_at) VALUES
('RF-FR260925-0007', 7, 7, 45000, 'STORE_CANCELLED', 'PENDING', 3, '2026-09-25 13:20:00');

INSERT INTO reviews (order_id, customer_id, store_id, rating, comment, created_at) VALUES
(1, 8, 1, 5, 'Burger còn nóng, bánh mềm. Giá giải cứu quá rẻ, ủng hộ mô hình chống lãng phí!', '2026-09-23 21:40:00'),
(2, 9, 1, 4, 'Ngon, nhưng 1 cái burger hơi nguội.', '2026-09-23 21:50:00');

INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, amount, detail, created_at) VALUES
(1, 'STORE_APPROVED', 'STORE', 1, NULL, 'Duyệt cửa hàng An Phát, phí 15%', '2026-09-01 09:00:00'),
(1, 'STORE_APPROVED', 'STORE', 2, NULL, 'Duyệt cửa hàng Sweet Paris, phí 15%', '2026-09-02 10:00:00'),
(1, 'STORE_APPROVED', 'STORE', 3, NULL, 'Duyệt cửa hàng Cơm Tấm 365, phí 15%', '2026-09-03 14:00:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 3, 22000, 'VNPay 14567003', '2026-09-23 18:01:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 1, 80000, 'VNPay 14567001', '2026-09-23 19:16:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 2, 120000, 'VNPay 14567002', '2026-09-23 19:31:00'),
(6, 'ORDER_COMPLETED', 'ORDER', 1, 80000, 'Giao hàng mã K7Q2M9', '2026-09-23 20:45:00'),
(6, 'ORDER_COMPLETED', 'ORDER', 2, 120000, 'Giao hàng mã P3X8D1', '2026-09-23 21:10:00'),
(NULL, 'ORDER_NO_SHOW', 'ORDER', 3, 22000, 'Quá giờ nhận 30 phút', '2026-09-23 21:30:00'),
(1, 'PAYOUT_CREATED', 'PAYOUT', 1, 170000, 'Kỳ 21-24/09, 2 đơn', '2026-09-25 09:00:00'),
(1, 'PAYOUT_PAID', 'PAYOUT', 1, 170000, 'Chuyển khoản VCB-FT26267112233', '2026-09-25 09:30:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 4, 87000, 'VNPay 14567004', '2026-09-25 10:21:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 5, 50000, 'VNPay 14567005', '2026-09-25 11:46:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 6, 60000, 'VNPay 14567006', '2026-09-25 12:11:00'),
(NULL, 'PAYMENT_SUCCESS', 'ORDER', 7, 45000, 'VNPay 14567007', '2026-09-25 13:01:00'),
(3, 'ORDER_CANCELLED', 'ORDER', 7, 45000, 'Khách tại quầy đã mua hết', '2026-09-25 13:20:00'),
(3, 'REFUND_REQUESTED', 'REFUND', 1, 45000, 'RF-FR260925-0007, lý do STORE_CANCELLED', '2026-09-25 13:20:00');

```

Phụ lục B. Câu truy vấn kiểm tra dữ liệu

File đi kèm: `foodrescue_checks.sql`. Xem giải thích ở mục 5.10.

```
-- =====
-- FOODRESCUE - Kiểm tra tính đúng của dữ liệu (chạy sau khi test / trước khi demo)
-- Mỗi câu kiểm tra trả về các dòng VI PHẠM. Dữ liệu đúng => mọi câu C1..C12 trả về 0 dòng,
-- câu C0 (đối soát tổng) phải có chenh_lech = 0, câu C13 chỉ để xem số liệu từng cửa hàng.
-- Nhóm: tiền (C0, C4-C9, C12) · kho hàng (C1, C1b, C1c, C2) · đơn và giỏ hàng (C3-C3d, C5, C11) · đánh giá (C10)
-- =====
USE foodrescuedb;

-- C0. Đối soát tổng: tiền thu từ khách = hoàn + chờ hoàn + đã chi + chờ chi + nợ cửa hàng + phí + đơn chưa giao
SELECT t.*, tien_thu - (da_hoan + cho_hoan + da_chi + cho_chi + no_cua_hang + phi_nen_tang + don_chua_giao) AS chenh_lech
FROM (
  SELECT
    (SELECT COALESCE(SUM(amount), 0) FROM payments WHERE status = 'SUCCESS') AS tien_thu,
    (SELECT COALESCE(SUM(amount), 0) FROM refunds WHERE status = 'COMPLETED') AS da_hoan,
    (SELECT COALESCE(SUM(amount), 0) FROM refunds WHERE status = 'PENDING') AS cho_hoan,
    (SELECT COALESCE(SUM(net_amount), 0) FROM payouts WHERE status = 'PAID') AS da_chi,
    (SELECT COALESCE(SUM(net_amount), 0) FROM payouts WHERE status = 'PENDING') AS cho_chi,
    (SELECT COALESCE(SUM(store_earning), 0) FROM orders WHERE status IN ('COMPLETED', 'NO_SHOW') AND payout_id IS NULL) AS
no_cua_hang,
    (SELECT COALESCE(SUM(commission_amount), 0) FROM orders WHERE status IN ('COMPLETED', 'NO_SHOW')) AS
phi_nen_tang,
    (SELECT COALESCE(SUM(total_amount), 0) FROM orders WHERE status = 'CONFIRMED') AS
don_chua_giao
) t;

-- C1. Tồn kho: quantity - remaining phải bằng tổng số túi trong các đơn còn hiệu lực
SELECT b.id, b.quantity, b.remaining_quantity,
       COALESCE(SUM(CASE WHEN o.status IN ('PENDING_PAYMENT', 'CONFIRMED', 'COMPLETED', 'NO_SHOW') THEN i.quantity END), 0) AS
da_ban
FROM rescue_batches b
LEFT JOIN order_items i ON i.batch_id = b.id
LEFT JOIN orders o ON o.id = i.order_id
GROUP BY b.id
HAVING b.quantity - b.remaining_quantity <> da_ban;

-- C1b. Số kho khớp tồn kho: tổng mọi biến động = remaining; tổng LISTED + ADDED + REMOVED = quantity;
-- dòng cuối cùng có remaining_after = remaining
SELECT b.id, b.quantity, b.remaining_quantity,
       COALESCE(SUM(m.change_qty), 0) AS tong_bien_dong,
       COALESCE(SUM(CASE WHEN m.reason IN ('LISTED', 'ADDED', 'REMOVED') THEN m.change_qty END), 0) AS tong_dua_len,
       (SELECT m2.remaining_after FROM stock_movements m2 WHERE m2.batch_id = b.id ORDER BY m2.created_at DESC, m2.id DESC LIMIT
1) AS remaining_cuoi
FROM rescue_batches b LEFT JOIN stock_movements m ON m.batch_id = b.id
GROUP BY b.id
HAVING tong_bien_dong <> b.remaining_quantity OR tong_dua_len <> b.quantity OR remaining_cuoi <> b.remaining_quantity;

-- C1c. Số kho khớp đơn: mỗi túi trong đơn có đúng 1 dòng ORDERED cùng số lượng; đơn đã EXPIRED/CANCELLED có dòng RETURNED
SELECT * FROM (
  SELECT i.order_id, i.batch_id, i.quantity, o.status,
         (SELECT -SUM(change_qty) FROM stock_movements m WHERE m.order_id = i.order_id AND m.batch_id = i.batch_id AND m.reason =
'ORDERED') AS da_tru,
         (SELECT SUM(change_qty) FROM stock_movements m WHERE m.order_id = i.order_id AND m.batch_id = i.batch_id AND m.reason =
'RETURNED') AS da_tra
  FROM order_items i JOIN orders o ON o.id = i.order_id
) x
WHERE da_tru IS NULL OR da_tru <> quantity
      OR (status IN ('EXPIRED', 'CANCELLED') AND (da_tra IS NULL OR da_tra <> quantity))
      OR (status NOT IN ('EXPIRED', 'CANCELLED') AND da_tra IS NOT NULL);

-- C2. Túi hết hàng phải ở trạng thái SOLD_OUT (hoặc đã hết hạn / huỷ)
SELECT id, remaining_quantity, status FROM rescue_batches
WHERE (remaining_quantity = 0 AND status = 'AVAILABLE') OR (remaining_quantity > 0 AND status = 'SOLD_OUT');

-- C3. Mọi túi trong đơn phải thuộc cửa hàng của đơn
SELECT o.id, o.store_id, b.id AS batch_id, b.store_id AS batch_store
FROM orders o JOIN order_items i ON i.order_id = o.id JOIN rescue_batches b ON b.id = i.batch_id
WHERE o.store_id <> b.store_id;

-- C3b. Tổng đơn = tổng chi tiết; số túi = tổng số lượng chi tiết; đơn nào cũng có ít nhất 1 dòng chi tiết
SELECT o.id, o.total_amount, SUM(i.subtotal) AS tong_chi_tiet, o.item_count, SUM(i.quantity) AS so_tui
FROM orders o LEFT JOIN order_items i ON i.order_id = o.id
GROUP BY o.id
HAVING COUNT(i.id) = 0 OR o.total_amount <> SUM(i.subtotal) OR o.item_count <> SUM(i.quantity);
```

```

-- C3c. Khung giờ lấy của đơn phải nằm trong khung giờ của mọi túi trong đơn
SELECT o.id, o.pickup_start, o.pickup_end, b.id AS batch_id, b.pickup_start AS b_start, b.pickup_end AS b_end
FROM orders o JOIN order_items i ON i.order_id = o.id JOIN rescue_batches b ON b.id = i.batch_id
WHERE o.pickup_start < b.pickup_start OR o.pickup_end > b.pickup_end;

-- C3d. Giỏ hàng: mỗi khách chỉ chứa túi của 1 cửa hàng
SELECT c.customer_id, COUNT(DISTINCT b.store_id) AS so_cua_hang
FROM cart_items c JOIN rescue_batches b ON b.id = c.batch_id
GROUP BY c.customer_id
HAVING COUNT(DISTINCT b.store_id) > 1;

-- C4. Đơn đã trả tiền (PAID / REFUND_*) phải có đúng 1 giao dịch SUCCESS bằng đúng số tiền; đơn UNPAID thì không có
SELECT o.id, o.payment_status, o.total_amount, COUNT(p.id) AS so_gd_thanh_cong, COALESCE(SUM(p.amount), 0) AS tong_gd
FROM orders o LEFT JOIN payments p ON p.order_id = o.id AND p.status = 'SUCCESS'
GROUP BY o.id
HAVING (o.payment_status <> 'UNPAID' AND (COUNT(p.id) <> 1 OR SUM(p.amount) <> o.total_amount))
OR (o.payment_status = 'UNPAID' AND COUNT(p.id) > 0);

-- C5. Trạng thái thanh toán phải khớp trạng thái đơn
SELECT id, status, payment_status FROM orders
WHERE (status IN ('CONFIRMED', 'COMPLETED', 'NO_SHOW') AND payment_status <> 'PAID')
OR (status IN ('PENDING_PAYMENT', 'EXPIRED') AND payment_status <> 'UNPAID')
OR (status = 'CANCELLED' AND payment_status = 'PAID');

-- C6. Đơn REFUND_PENDING / REFUNDED phải có đúng 1 bản ghi hoàn tiền tương ứng, số tiền bằng total_amount
SELECT o.id, o.payment_status, r.status AS refund_status, o.total_amount, r.amount
FROM orders o LEFT JOIN refunds r ON r.order_id = o.id
WHERE (o.payment_status = 'REFUND_PENDING' AND (r.id IS NULL OR r.status <> 'PENDING' OR r.amount <> o.total_amount))
OR (o.payment_status = 'REFUNDED' AND (r.id IS NULL OR r.status <> 'COMPLETED' OR r.amount <> o.total_amount))
OR (o.payment_status IN ('UNPAID', 'PAID') AND r.id IS NOT NULL);

-- C7. Kỳ chi trả: số đơn và số tiền phải bằng tổng của đúng các đơn gắn vào kỳ; đơn phải cùng cửa hàng
SELECT p.id, p.order_count, COUNT(o.id) AS so_don, p.gross_amount, SUM(o.total_amount) AS gross,
p.commission_amount, SUM(o.commission_amount) AS phi, p.net_amount, SUM(o.store_earning) AS net
FROM payouts p LEFT JOIN orders o ON o.payout_id = p.id
WHERE p.status <> 'CANCELLED'
GROUP BY p.id
HAVING p.order_count <> COUNT(o.id) OR p.gross_amount <> SUM(o.total_amount)
OR p.commission_amount <> SUM(o.commission_amount) OR p.net_amount <> SUM(o.store_earning)
OR SUM(o.store_id <> p.store_id) > 0;

-- C8. Kỳ chi trả đã huỷ không được còn đơn gắn vào
SELECT p.id, COUNT(o.id) FROM payouts p JOIN orders o ON o.payout_id = p.id WHERE p.status = 'CANCELLED' GROUP BY p.id;

-- C9. Đơn trong kỳ chi trả phải có ngày chốt đơn nằm trong kỳ
SELECT o.id, o.completed_at, p.period_from, p.period_to FROM orders o JOIN payouts p ON p.id = o.payout_id
WHERE DATE(o.completed_at) NOT BETWEEN p.period_from AND p.period_to;

-- C10. Đánh giá phải thuộc đơn COMPLETED, đúng khách và đúng cửa hàng của đơn
SELECT r.id FROM reviews r JOIN orders o ON o.id = r.order_id
WHERE o.status <> 'COMPLETED' OR r.customer_id <> o.customer_id OR r.store_id <> o.store_id;

-- C11. Người giao hàng phải là chủ hoặc nhân viên của đúng cửa hàng
SELECT o.id, o.handled_by FROM orders o
JOIN users u ON u.id = o.handled_by
JOIN stores s ON s.id = o.store_id
WHERE NOT ((u.role = 'STORE_OWNER' AND s.owner_id = u.id) OR (u.role = 'STORE_STAFF' AND u.store_id = s.id));

-- C12. Mọi giao dịch tiền quan trọng phải có dòng nhật ký
SELECT 'PAYMENT_SUCCESS' AS thieu_log, p.order_id AS id FROM payments p
WHERE p.status = 'SUCCESS'
AND NOT EXISTS (SELECT 1 FROM audit_logs a WHERE a.action = 'PAYMENT_SUCCESS' AND a.entity_type = 'ORDER' AND a.entity_id =
p.order_id)
UNION ALL
SELECT 'REFUND_REQUESTED', r.id FROM refunds r
WHERE NOT EXISTS (SELECT 1 FROM audit_logs a WHERE a.action = 'REFUND_REQUESTED' AND a.entity_type = 'REFUND' AND a.entity_id =
r.id)
UNION ALL
SELECT 'PAYOUT_PAID', p.id FROM payouts p
WHERE p.status = 'PAID'
AND NOT EXISTS (SELECT 1 FROM audit_logs a WHERE a.action = 'PAYOUT_PAID' AND a.entity_type = 'PAYOUT' AND a.entity_id =
p.id);

-- C13. Điểm đánh giá và doanh thu từng cửa hàng (xem, không phải kiểm tra lỗi)
SELECT s.id, s.name,
(SELECT ROUND(AVG(rating), 2) FROM reviews r WHERE r.store_id = s.id) AS diem_tb,
(SELECT COALESCE(SUM(total_amount), 0) FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED', 'NO_SHOW'))
AS doanh_thu,
(SELECT COALESCE(SUM(commission_amount), 0) FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED', 'NO_SHOW'))
AS phi,
(SELECT COALESCE(SUM(store_earning), 0) FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED', 'NO_SHOW'))

```

```
AS thuc_nhan,  
    (SELECT COALESCE(SUM(net_amount), 0)      FROM payouts p WHERE p.store_id = s.id AND p.status = 'PAID')  
AS da_chi,  
    (SELECT COALESCE(SUM(store_earning), 0)    FROM orders o WHERE o.store_id = s.id AND o.status IN ('COMPLETED','NO_SHOW')  
AND o.payout_id IS NULL) AS con_no  
FROM stores s;
```